



Groupe Hospitalier
Artois-Ternois

Rapport de stage :

Administrateurs systèmes et réseaux

DUVAL Baptiste

7 Avril 2025 au 30 Mai 2025

Tuteur en entreprise : DUPRE Mathieu

Tuteur académique : LESTOQUOY Laura

Organisme d'accueil : Centre Hospitalier d'ARRAS

3 Bd Georges Besnier, 62000 Arras

Etablissement / Formation : IUT de Béthune / BUT Réseaux et télécommunications

1230 Rue de l'Université, 62400 Béthune

Remerciements

Je souhaiterais tout d'abord, adresser mes remerciements à l'IUT de Béthune et ses enseignants pour les compétences que j'ai pu acquérir tout au long de l'année grâce à leur enseignement et pour m'avoir permis d'effectuer un stage au sein du service informatique du Groupe Hospitalier Artois Ternois.

Je remercie tout particulièrement Monsieur Mathieu Dupré, mon tuteur de stage, Monsieur Patrick Majka responsable du système d'information, Monsieur Jean-Baptiste Cordier et Monsieur Alexis Paradis et toute l'équipe du service informatique, pour m'avoir accueilli, accordé leur confiance et transmis un grand nombre de leurs connaissances tout au long de ces 8 semaines de stage.

Je tiens à remercier aussi toutes les personnes qui m'ont aidé dans ma recherche de stage, toutes les entreprises qui ont pris le temps de répondre à mes demandes de stage.

Je tiens à remercier mes parents pour la relecture de ce rapport de stage.

Table des matières

Remerciements.....	2
Table des matières.....	3
Introduction.....	4
Présentation du service et du Groupe Hospitalier Artois Ternois.....	5
I. Le Groupe Hospitalier Artois Ternois.....	5
II. Le service informatique du Groupe Hospitalier Artois-ternois.....	6
Présentation de mes missions.....	7
I. Dépannage sur caméras.....	7
II. Câblage de caméras sur switch.....	7
III. Réglage de tablette.....	8
IV. Récupération identifiant de téléphone.....	8
V. Gestion d'une baie.....	8
VI. Changement d'une borne WIFI.....	9
VII. Paramétrage de téléphones professionnels.....	9
VIII. Restauration de WIFI sur appareil médical.....	9
IX. Mise en place du service Stirling-PDF.....	10
X. Dépannage de télévisions.....	10
XI. Mise en place de rConfig.....	11
Présentation de mes projets.....	12
I. Script création des utilisateurs	12
II. Centralisation et supervision des Logs.....	24
III. Création d'un serveur de gestion des logs CDR.....	34
Conclusion.....	47
Outils utilisé lors du stage.....	48
Source :	49
Annexe :	51

Introduction

Ce stage est réalisé dans le cadre de l'obtention de mon Bachelor Universitaire de Technologie Réseaux & Télécommunications à l'Université d'Artois, plus précisément à l'Institut Universitaire de Béthune. J'ai réalisé ce stage durant la période du 7 avril 2025 au 30 mai 2025 (une période d'environ deux mois) au sein du service informatique du GHAT (Groupe Hospitalier Artois-Ternois). Ce stage a pour but de m'initier à la vie professionnelle, de me former et de mettre mes compétences au service du service informatique.

Au cours de ma formation à l'Université d'Artois, j'ai eu l'opportunité de me familiariser avec les différents aspects de l'informatique, tels que la configuration d'équipements réseau, la mise en place d'un serveur web, le développement de services web...

Parmi les possibilités qui m'ont été données, j'ai choisi de rejoindre le service informatique du GHAT pour plusieurs raisons que je préciserais ci-dessous.

À travers ce rapport, vous prendrez connaissance des activités du service au sein des hôpitaux du groupe et des missions que nous pouvons effectuer. Ce stage m'a confirmé que le poste d'Administrateur système et réseaux est un poste qui me plaît, qui me permet de m'épanouir dans le monde du travail, tant les missions et les projets peuvent varier. J'aimerais poursuivre dans ce genre de poste pour le reste de mes études. De plus, les missions étant très variées, elles m'ont permis d'acquérir de nouvelles compétences et connaissances, tout en développant et en pouvant mettre à disposition les compétences que j'ai acquises lors de ces deux années de BUT Réseaux et télécommunications.

Les missions qui m'ont été confiées peuvent aller de la restauration de tablette car problème de stockage à la réparation sur site d'une borne wifi Cisco.

Certaines missions et projets correspondent à des projets que nous pouvons réaliser au sein de l'IUT, ce qui m'a permis d'utiliser des compétences et des connaissances que j'ai acquises lors de ces cours magistraux et travaux pratiques.

Présentation du service et du Groupe Hospitalier Artois Ternois

I. Le Groupe Hospitalier Artois Ternois

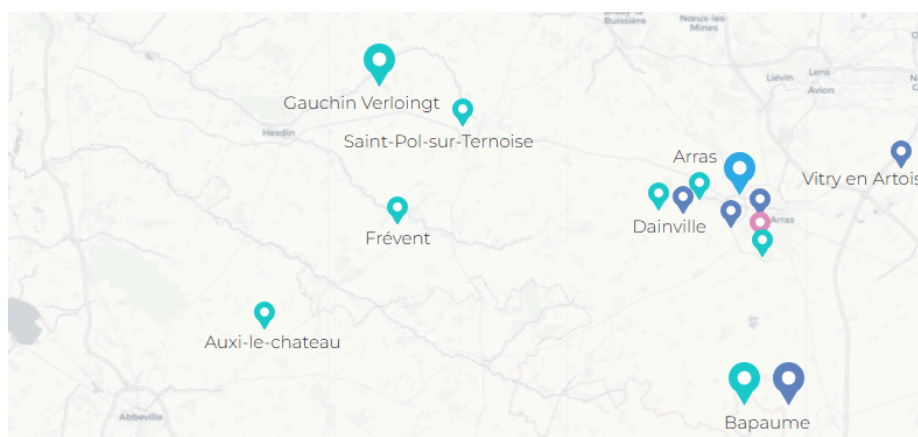
Tout d'abord le Centre hospitalier d'Arras est un établissement public offrant une réponse aux besoins de santé d'un territoire de plus de 300 000 personnes, en plein cœur de l'Artois.

Il offre à la population une large gamme d'activités et de ressources pour le diagnostic, les soins, la prévention et l'éducation à la santé. Il couvre ainsi la quasi-totalité des disciplines de médecine, de chirurgie et d'obstétrique. Il dispose d'une maternité et d'une réanimation néonatale de niveau III et assure également une activité de psychiatrie et de santé mentale pour adultes, enfants et adolescents.



Image du Centre Hospitalier d'Arras

Le Centre Hospitalier d'Arras est depuis 2016 groupé avec les centres hospitaliers de l'Artois et du Ternois qui sont situés dans plus de 8 villes différentes (Arras, Bapaume, Auxi-le-Château, Dainville, Frévent, Gauchin Verloingt, Saint-Pol-sur-Ternoise et Vitry-en-Artois).



Centres Hospitalier présent dans le GHAT

Concrètement, ce regroupement permet aux hôpitaux membres de partager des fonctions clés comme la gestion des systèmes d'information, les achats, la formation, ou encore les ressources humaines. Le GHAT favorise aussi une meilleure complémentarité entre les établissements : certains se spécialisent dans des domaines précis (médecine, chirurgie, gériatrie...), tandis que d'autres assurent des soins de proximité.

L'objectif principal du GHAT est donc d'optimiser l'organisation des soins, tout en garantissant un accès équitable à la santé pour tous les habitants du territoire, notamment dans les zones rurales ou sous-dotées. Cette mutualisation permet également de faire face aux contraintes budgétaires tout en maintenant un haut niveau de qualité dans les services rendus.

II. Le service informatique du Groupe Hospitalier Artois-ternois

Le service informatique s'occupe de tous les hôpitaux du groupe, ils gèrent toutes les demandes sous forme de tickets. Les tickets sont la principale source de demande de gestion de problèmes pour les équipes informatiques.

Le service informatique est basé à Arras, il est composé de 3 pôles. Il y a le pôle de poste de travail, il s'agit d'une équipe composée de 5 personnes qui s'occupe de l'installation et du déploiement des ordinateurs et des imprimantes au sein de l'hôpital. Donc quand une personne a besoin d'une nouvelle imprimante ou d'un nouvel ordinateur, c'est cette équipe qui s'occupe de tout cette partie informatique. Ils s'occupent aussi de la partie de création de comptes Active Directory.

Ensuite, je faisais partie du pôle réseau composé de trois personnes, dans ce pôle cette équipe s'occupe du côté infrastructure réseau, donc déploiement de switch, de routeur, configuration IP, mise en place de pare-feu. Ils s'occupent aussi du déploiement des serveurs, la gestion Active Directory donc gestion des droits des GPO, ils s'occupent de la partie sécurité de l'hôpital. Ils font aussi de la configuration de téléphones et de bornes WIFI.

Et le dernier pôle, le pôle applicatif, est composé de 6 personnes dans ce bureau ils s'occupent de la partie mise en place et paramétrages des logiciels utilisés dans l'hôpital.

Ce service est donc composé de 14 personnes, qui gèrent l'entièreté de la partie informatique du groupe.

Présentation de mes missions

Durant ce stage j'ai effectué un certain nombre de missions, que j'ai effectué seul ou en équipe.

I. Dépannage sur caméras

Nous sommes allés en équipe dépanner un service qui n'arrivait plus à accéder au flux vidéo des caméras. Nous sommes donc allés vérifier sur les ordinateurs si nous pouvions accéder aux caméras une par une. Comme nous pouvions accéder aux caméras grâce aux IP nous en avons déduit que le problème venait du script qui servait à afficher les flux des caméras sur VLC. Après avoir modifié le script, les caméras ont été affichées sur VLC et tout est revenu à la normale.

II. Câblage de caméras sur switch

Après rénovation d'un bâtiment, des caméras ont été installées, donc nous sommes allés en équipes brancher les caméras au switch de ce bâtiment. Après que les caméras aient bien été branchées au switch nous avons connecté ce switch au bâtiment principal, ces switchs sont branchés entre eux grâce à la fibre, le type de fibre utilisé est une fibre multimode.



Switch connecté avec la fibre

III. Réglage de tablette

La tablette d'un membre du personnel rencontrait un problème de stockage, la tablette indiquait continuellement que le stockage était plein alors qu'aucune application n'était installée. Après des recherches sur le modèle de la tablette, aucune personne ne rencontrait ce problème, donc comme la tablette était vierge de tous documents, applications ou photos. J'ai décidé de la réinstaller pour voir si le problème allait se résoudre. Après réinitialisation j'ai remarqué que le stockage a été remis à 0, le problème a donc été résolu.

IV. Récupération identifiant de téléphone

Après réception d'équipements, il m'a été confié la mission de scanner des numéros d'identifiants des téléphones, pour les mettre dans un tableau et pour vérifier si les équipements demandés étaient bien présents.

V. Gestion d'une baie

Avant la rénovation d'un bâtiment comportant d'anciens bureaux, nous sommes allés noter le numéro des prises RJ45 car, comme des murs allaient être abattus, il y avait un risque de dysfonctionnement du switch. Après avoir noté le numéro des prises, nous sommes allés à la baie pour débrancher les câbles correspondant aux prises. Il n'y a donc pas de risque qu'un switch ne fonctionne plus à cause d'un câble coupé lors de la casse du mur.

VI. Changement d'une borne WIFI

Après avoir été contacté pour une borne WIFI défectueuse, nous nous sommes rendus sur place pour vérifier le problème. Nous avons pris une borne et un câble RJ45 de rechange car nous savions que cette borne se trouvait dans une cuisine et quand cette cuisine était nettoyée de l'eau peut l'atteindre et donc oxydé le câble et la borne. Après avoir trouvé la borne, nous l'avons débranché, nous avons pu voir que la prise RJ45 était oxydé, nous l'avons donc changée.



Câble oxydé et rouillé

Après avoir vu que cela ne fonctionnait toujours pas nous avons changé le câble et nous en avons mis un neuf et la borne fonctionnait. Il ne manquait plus qu'à la paramétrer et la borne était opérationnelle.

VII. Paramétrage de téléphones professionnels

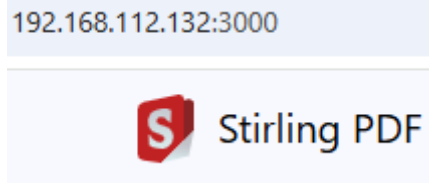
Avant de distribuer les téléphones professionnels au personnel, il faut les paramétrer. Pour ce faire, au démarrage du téléphone, nous pouvons choisir de le passer en mode professionnel. Après avoir effectué cela, nous les connectons avec leurs comptes professionnels, puis nous les intégrons au domaine téléphonique. Ensuite, une fois qu'ils sont bien intégrés au domaine et paramétrés en mode professionnel, nous effectuons toutes les mises à jour des téléphones, et ils sont enfin prêts à être distribués.

VIII. Restauration de WIFI sur appareil médical

Après un ticket généré, une personne issue du personnel médical nous a amené du matériel utilisé médicalement. Il m'a donc été confié de restaurer la connexion WIFI sur ces appareils. Ces appareils étaient des casques VR, j'ai donc allumé et enfilé le casque, après m'être adapté à son utilisation je suis allé dans les paramètres pour rétablir la WIFI sur les appareils.

IX. Mise en place du service Stirling-PDF

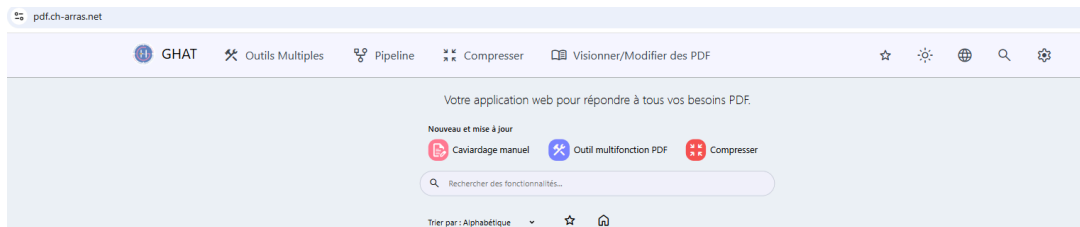
Il m'a été confié de mettre en place un service Stirling-PDF, pour pouvoir comprendre comment cela fonctionne et vérifier qu'il n'y ait pas de failles majeures. Je me suis donc renseigné sur internet pour pouvoir installer Stirling-PDF sur une machine Debian. Quand j'ai trouvé le site officiel de Stirling-PDF, j'ai donc réalisé toutes les étapes d'installation du service.



Site internet Stirling PDF

Après avoir installé le service, j'ai donc navigué sur le site pour comprendre comment cela fonctionne, quels types de modifications il peut faire sur un PDF, comment il convertit des fichiers en fichiers PDF. Quand j'ai compris les fonctionnalités de ce site, je suis donc allé me renseigner sur les failles majeures éventuelles, après avoir parcouru les forums, j'en ai conclu qu'aucune faille majeure n'était présente.

Ensuite je l'ai développé et installé sur un serveur, et l'ai lié à Nginx pour pouvoir avoir accès au site grâce au lien : pdf.ch-arras.net. J'ai ensuite effectué des modifications sur les pages HTML du site pour pouvoir y mettre le logo et le nom de l'hôpital.



Le site après toutes les modifications

X. Dépannage de télévisions

Suite à plusieurs plaintes sur deux télévisions défectueuses dans des chambres des patients, nous nous sommes rendus sur place pour comprendre ce qu'il se passait. Nous en avons déduit que la prise RJ45 sur les télévisions était très abîmée, nous avons donc remplacé les deux télévisions par des nouvelles. Nous les avons branchées au réseau et toutes les chaînes sont revenues sans problèmes.

XI. Mise en place de rConfig

Il m'a été confié de mettre en place le service docker rConfig, pour comprendre et savoir comment s'en servir. Cet outil peut-être une alternative des outils utilisés dans le service informatique. C'est un outil de supervision des équipements réseaux comme les switchs, les routeurs et les firewall. Je me suis donc servi de la documentation officielle de rConfig pour l'installer et pour créer des configurations et savoir comment envoyer les configurations et les logs des équipements sur le service.

Présentation de mes projets

I. Script création des utilisateurs

Pourquoi faire cela ?

L'objectif principal de ce projet est de gérer et d'optimiser la création des comptes sur Active Directory.

La gestion des utilisateurs dans l'Active Directory est très importante dans le cadre d'un établissement hospitalier, surtout pour garantir l'attribution des droits d'accès aux fichiers partagés, aux applications et aux serveurs. Mais, la création manuelle de comptes et de leurs droits peut être susceptible aux erreurs.

Les erreurs peuvent avoir de grosses conséquences sur la sécurité, en permettant à un groupe non autorisé d'accéder à des fichiers ou dossiers non autorisés. De même, la gestion manuelle des utilisateurs peut entraîner une inefficacité organisationnelle, notamment par des délais de mise en œuvre et un suivi difficile des informations. Ainsi l'automatisation de ce processus via un script dédié permet de réduire les risques, mais aussi d'éviter une grosse perte de temps.

Présentation du script

Un fichier Excel comportant des informations sur du personnel m'a été transmis, mon but avec ce fichier était d'automatiser avec un script PowerShell la création des utilisateurs dans l'Active Directory.

J'ai donc transformé ce fichier xlsx en fichier csv avec des séparateurs en « ; » pour pouvoir utiliser les colonnes en champs que je voulais utiliser.

Ces colonnes comportaient plusieurs informations comme le nom, le prénom, la fonction, le bureau, les matricules, le grade, la date expiratoire et d'autres informations pour la création de son lecteur personnel.

J'ai donc commencé par créer un Windows serveur avec un domaine, c'est donc dans ce domaine que les utilisateurs seront créés et stockés.

Quand mon domaine a été créé j'ai donc commencé à écrire le script qui allait automatiser la création des utilisateurs. Dans ce script, j'ai commencé par définir le chemin complet du fichier « compte.csv » puis le stocker dans une variable « \$CSVFile ». Puis je lis mon fichier chargeant en mémoire toutes les lignes du fichier dans la variable « \$CSVData » sous forme de tableau car nous séparons les caractères avec « ; ».

Ensuite, je définis une variable appelée \$Groupe1 et je lui assigne la valeur "groupe1", ce qui correspond au nom du groupe auquel je souhaite ajouter des utilisateurs. Mettre le groupe dans une variable permet de rendre le script plus flexible et plus facile à maintenir, car je n'ai qu'à modifier la

valeur de la variable si le nom du groupe venait à changer. Ensuite, j'utilise cette variable pour ajouter les utilisateurs à ce groupe, ce qui évite de répéter le nom du groupe dans plusieurs endroits du script.

Quand les groupes sont dans des variables, j'initialise mes fonctions pour le fichier log. J'ai trois fonctions, une Erreur qui affichera en rouge dans le terminal quand il y aura une erreur, en jaune les warnings et en vert quand tout se passe bien. Cette fonction écrit aussi dans le fichier de log avec [ERROR] quand la ligne est dans cette fonction.

Exemple :

```
function Error {  
    param ([string]$message)  
    Write-Host $message -ForegroundColor Red  
    Add-Content -Path $LogFile -Value "[ERROR] $message"  
}
```

```
Error "Ligne $index : Le champ NOM est vide"
```

J'ai aussi une d'autres fonction qui me permettent d'écrire des commandes dans des fichiers, comme certaines commandes ne peuvent pas être exécutée sur mon serveur, j'ai créé une fonction qui écrira les commandes de chaque utilisateur dans un fichier, il y aura plus qu'à faire un copier-coller.

Exemple :

```
function mail {  
    param ([string]$message)  
    Add-Content -Path $cheminMail -Value $message  
}
```

```
mail "Enable-Mailbox -Identity '$UtilisateurMatricule' -Alias '$UtilisateurPrenom.$UtilisateurNom' -Database '$UtilisateurBDD'"
```

Ensuite grâce à la boucle foreach nous répétons une action pour chaque élément dans « \$CSVData ». Donc ici pour chaque élément dans la liste [\\$CSVData](#), fais ce qui suit avec cet élément que je vais appeler \$Utilisateur. Ensuite je vais chercher les informations de l'utilisateur courant dans la ligne CSV, en me servant du nom des colonnes du fichier.

Par exemple si je veux le nom de l'utilisateur dans le fichier :

```
foreach ($Utilisateur in $CSVData) {  
    $index++  
    $UtilisateurNom = $Utilisateur.NOM  
    $UtilisateurPrenom = $Utilisateur.Prenom
```

Ici nous avons donc \$Utilisateur est l'objet courant, nous y ajoutons «. NOM » qui va chercher la colonne « NOM » le nom de l'utilisateur courant. Je viens insérer cette valeur dans la variable \$UtilisateurNom. Je fais donc cela avec toutes les colonnes présentes dans le fichier.

Après avoir créé la boucle je vais pouvoir faire des vérifications d'erreurs, c'est-à-dire de vérifier si les champs obligatoires sont bien remplis, s'ils ne sont pas remplis une erreur est enregistrée dans la variable « erreursTrouvees ». A la fin de la partie de vérification une question est posée, elle nous demande si oui ou non nous voulons créer les utilisateurs malgré les erreurs, et si aucune erreur n'a été détectée le script de création se lance automatiquement.

```
$confirmation = "non"
if ($erreursTrouvees) {
    $confirmation = Read-Host "Des erreurs ont été détectées. Voulez-vous malgré tout créer les utilisateurs ? (oui/non)"
}

if (-not $erreursTrouvees -or $confirmation -eq "oui") {
    Write-Host "Lancement du script de création des utilisateurs..."
}
```

Quand l'utilisateur dit oui nous relançons une boucle foreach et remettons les informations de colonnes dans des variables pour pouvoir créer les utilisateurs. Mais avant de les créer nous supprimons dans les noms et dans les prénoms les espaces, pour éviter les erreurs lors de la création de l'adresse mail et de la boîte mail. Ensuite nous modifions le prénom des utilisateurs pour qu'il soit au bon format c'est-à-dire « Prénom », donc quand tout est en majuscule, en minuscule une modification est effectuée. Puis nous mettons le mail dans la variable UtilisateurPN :

```
$UtilisateurPN = "$UtilisateurPrenom.$UtilisateurNom@Stage.fr"
```

Après avoir fait cela nous faisons de nouvelles vérifications, mais si une nouvelle erreur est détectée le script passera la ligne csv de l'utilisateur et il ne sera pas créé. Par exemple s'il manque le nom, le prénom ou n'importe quel champ important pour un utilisateur il ne sera pas créé. Il y a aussi une vérification du format de certains champs, par exemple le matricule, le script vérifie avec un regex qu'il est bien aux normes et qu'il n'y a pas de caractères spéciaux. Après les vérifications de format, nous faisons des vérifications que l'utilisateur existe ou non dans l'AD. Si l'utilisateur existe dans tout l'AD sauf dans l'OU à détruire, il sera mis à jour avec ses nouvelles informations. Et s'il existe dans l'OU à supprimer, il sera déplacé dans son nouvel OU, mis à jour avec ses nouvelles données, nous allons modifier son mot de passe avec le mot de passe de base de l'hôpital, forcer le changement de mot de passe à l'ouverture de session, réactivé et nous allons désactiver la carte puce, qui sert à bloquer l'ouverture de session.

Maintenant s'il n'existe pas et que les informations de l'utilisateur sont aux normes il sera créé, donc tout d'abord nous créons le profil avec la commande :

```
New-ADUser -GivenName "$UtilisateurPrenom" -Surname "$UtilisateurNom" -Name "$UtilisateurNom $UtilisateurPrenom" `
-DisplayName "$UtilisateurNom $UtilisateurPrenom" -SamAccountName "$UtilisateurMatricule" `
-Path $UtilisateurOU -UserPrincipalName "$UtilisateurPN" -Title "$UtilisateurFonction" `
-Department "$UtilisateurBureau" -Office "$UtilisateurBureau" -EmployeeID "$UtilisateurAGRIH"
```

Après avoir créé l'utilisateur avec tous ses attributs personnels et professionnels (comme le prénom, le nom, le matricule, le poste ou le bureau), ces informations nous permettent de gérer son identité dans l'annuaire Active Directory. Elles servent notamment à l'authentification, à l'affichage dans les outils collaboratifs (comme Outlook ou Teams), au classement dans les services.

Nous pouvons ensuite lui attribuer un mot de passe provisoire qu'il devra changer obligatoirement à sa première connexion.

```
Set-ADAccountPassword $UtilisateurMatricule -Reset -NewPassword (ConvertTo-SecureString -AsPlainText "Progrtr00++" -Force)
```

Cette commande nous permet de réinitialiser le mot de passe de l'utilisateur \$UtilisateurMatricule. Elle lui attribue automatiquement un mot de passe temporaire, ici "Progrtr00++", sans avoir besoin de connaître l'ancien mot de passe.

Le mot de passe est converti au format sécurisé via [ConvertTo-SecureString](#), ce qui garantit que les informations sensibles ne sont pas stockées en clair dans la mémoire PowerShell.

Ce mot de passe est donc appliqué à tous les utilisateurs créés automatiquement dans la boucle de création, ce qui permet d'automatiser le processus de création des comptes.

```
Set-ADUser $UtilisateurMatricule -ChangePasswordAtLogon $True
```

Cette commande complète la création en imposant à l'utilisateur de modifier ce mot de passe temporaire dès sa première connexion. Cela renforce la sécurité, car le mot de passe attribué automatiquement ne pourra pas être utilisé indéfiniment.

Ensuite, nous écrivons dans des fichiers les commandes de création des fichiers et les commandes de création de la boîte mail. Donc si le champ dossier de l'utilisateur est rempli, nous allons pouvoir écrire dans le fichier «**Commandes-CHAUSERS**» les commandes suivantes :

```
fichier "New-Item -Path 'E:\Users\$UtilisateurDossier\$UtilisateurMatricule' -ItemType Directory"
fichier "New-SmbShare -Name '$UtilisateurMatricule' -Path 'E:\Users\$UtilisateurDossier\$UtilisateurMatricule'
-ChangeAccess 'Tout le monde'"
fichier "Add-NTFSAccess -Path 'E:\Users\$UtilisateurDossier\$UtilisateurMatricule' -Account '$UtilisateurMatricule'
-AccessRights Modify"
Set-ADUser -Identity "$UtilisateurMatricule" `
    -HomeDirectory "\\chausers\$UtilisateurMatricule\$" `
    -HomeDrive "G:" `
    -ScriptPath "charras.vbs"
Info "Ajout effectuer avec succès au fichier Commandes-CHAUSERS-$(formattedDate.txt)"
```

Nous écrivons ces commandes dans un fichier différent car le serveur de fichier est un serveur distinct du serveur Windows Active Directory. Avec ces commandes, nous créons le dossier de l'utilisateur, nous créons le partage puis nous lui donnons les droits. Il lui sera aussi attribuer son dossier partagé, son répertoire personnel et un script de lancement de session (qui va mapper automatiquement son lecteur).

Par contre si le champ dossier de l'utilisateur est vide, cette étape sera donc ignorée. Le processus est similaire pour la création de la boîte mail sauf que le champ à remplir ici est le champ BDD.

Après cela, nous pouvons maintenant intégrer tous les utilisateurs dans leurs groupes. La ligne suivante permet d'ajouter chaque utilisateur, que l'on identifie grâce à la variable `$UtilisateurMatricule`, au groupe défini dans la variable `$Groupe1`.

```
Add-ADGroupMember -Identity $Groupe1 -Members "$UtilisateurMatricule"
```

Dans certains cas, les utilisateurs doivent être affectés à des groupes spécifiques en fonction de leur grade ou de leur rôle au sein de l'hôpital. Par exemple, si un utilisateur a pour grade *étudiant*, cette information est disponible dans une colonne dédiée de ses attributs, et elle est stockée dans une variable comme `$UtilisateurGrade = $Utilisateur.Grade`.

Pour mettre les utilisateurs au groupe approprié selon leur grade, nous utilisons une condition `if` pour vérifier la valeur de cette variable. Si le grade de l'utilisateur est égal à "étudiant", l'utilisateur sera ajouté au groupe "GroupeÉtudiant".

Voici un exemple de script qui effectue cette opération :

```
if ($UtilisateurGrade -eq "étudiant") {  
    Add-ADGroupMember -Identity $GroupeÉtudiant -Members "$UtilisateurMatricule"  
}
```

Ainsi, le script vérifie son grade et, en fonction de celui-ci, l'ajoute automatiquement au groupe correspondant. Cela permet d'automatiser le processus d'affectation des utilisateurs à leurs groupes, basé sur leur statut ou rôle au sein de l'organisation, comme par exemple étudiants, employés, administrateurs, etc.

Ensuite, nous avons une colonne avec la date d'expiration du compte utilisateur. Nous utilisons cette information pour déterminer si une date a été fournie, si elle est valide, et si elle peut être appliquée. Le script analyse et traite cette date, comme expliqué ci-dessous :

On utilise ici `[string]::IsNullOrEmpty()` pour détecter un champ vide ou mal renseigné. Si c'est le cas, on affiche un message d'avertissement (log *warning*) pour signaler que l'utilisateur n'a pas de date d'expiration définie.

```
if ([string]::IsNullOrEmpty($UtilisateurDate)) {  
    log "[WARNING] Aucune date d'expiration n'a été donnée pour $UtilisateurNom $UtilisateurPrenom."  
}
```

Si une date a bien été renseignée, le script essaie de la convertir au format date (dd/MM/yyyy). On fait cela avec `ParseExact`, qui vérifie si la date est au bon format. Par exemple, "23/04/2025" est valide, mais "2025-04-23" ne le sera pas ici.

```
else {  
    try {  
        $ParsedDate = [datetime]::ParseExact($UtilisateurDate, "dd/MM/yyyy", $null)
```

On compare ensuite cette date avec la date actuelle.

Si la date d'expiration est déjà dépassée, un message d'erreur est généré, car créer un compte avec une date déjà expiré ce n'est pas normal.

```
if ($Date -gt $ParsedDate) {  
    log "[ERROR] La date d'expiration pour $UtilisateurNom $UtilisateurPrenom est déjà passée : $ParsedDate"  
}
```

Si la date est valide et encore dans le futur, elle est appliquée à l'utilisateur via `Set-ADAccountExpiration`, qui définit la date d'expiration du compte dans Active Directory. Un log est ensuite généré pour confirmer l'ajout de cette date.

```
else {  
    Set-ADAccountExpiration -Identity $UtilisateurMatricule -Datetime $ParsedDate  
    log "Date d'expiration ajoutée : $ParsedDate"  
}  
}
```

Ce bloc de script permet de gérer automatiquement la date d'expiration des comptes utilisateurs. Pour chaque utilisateur, il vérifie si une date d'expiration est renseignée dans les données. Si c'est le cas, le script tente de la convertir au bon format (dd/MM/yyyy) et s'assure que la date est bien dans le futur. Si la date est au bon format, la date d'expiration est appliquée au compte grâce à la commande [Set-ADAccountExpiration](#). Cela permet, par exemple, de désactiver automatiquement un compte à une date précise.

Activation du compte utilisateur

Une fois que toutes les étapes précédentes sont faites, le script se termine en activant le compte de l'utilisateur dans Active Directory avec la commande :

```
Enable-ADAccount "$UtilisateurMatricule"
```

Cela permet de rendre le compte de l'utilisateur actif et prêt à être utilisé.

L'utilisateur est désormais créé et configuré avec son dossier personnel, ses droits d'accès, et son profil, prêt à se connecter.¹

Zoom et approfondissement de la vérification des champs :

Vérifications sur champs obligatoire :

Chaque champ important (comme le **nom**, **prénom**, **matricule**, **AGRIH**, ou **dossier**) est vérifié avec la même méthode. Le script utilise `IsNullOrEmpty()` pour détecter si le champ est vide ou ne contient que des espaces.

```
if ([string]::IsNullOrEmpty($UtilisateurNom)) {  
    Error "Ligne $index : Le champ NOM est vide (Matricule : $UtilisateurMatricule)."  
    $erreursTrouvees = $true  
    continue  
}
```

À chaque fois :

Un message d'erreur est inscrit dans le fichier log, la variable `$erreursTrouvees` est mise à true, et le script passe à l'utilisateur suivant avec `continue`. La logique est toujours la même, seule la variable testée change à chaque vérification.

Vérifications du format :

Cette partie du script me permet de vérifier si les champs importants et qui ont une certaine forme à avoir sont bien au bon format, qu'ils ne comprennent pas de caractères spéciaux, qu'ils commencent et finissent par les bons caractères...

```
if ($UtilisateurMatricule -notmatch "^[a-zA-Z0-9]+$") {  
    Error "Ligne $index : Le matricule $UtilisateurMatricule contient des caractères non autorisés."  
    $erreursTrouvees = $true  
    continue  
}
```

¹ Voir annexe pour captures d'écrans d'un utilisateur dans l'Active Directory, de son lecteur mappé, de sa session.

```

if ($UtilisateurMatricule -notmatch "^d{4,6}[a-zA-Z]{2,3}$") {
    Warning "Ligne $index : Le matricule $UtilisateurMatricule ne respecte pas le format attendu."
}

if ($UtilisateurBDD -ne "" -and $UtilisateurBDD -notmatch "^BDD") {
    Error "Ligne $index : Le champ BDD '$UtilisateurBDD' ne commence pas par 'BDD'."
    $erreursTrouvees = $true
    continue
}

if ($UtilisateurMatricule.Substring(0,5) -ne $UtilisateurAGRIH.Substring(0,5)) {
    Error "Ligne $index : Les 5 premiers caractères du matricule '$UtilisateurMatricule' ne correspondent pas à ceux de '$UtilisateurAGRIH'."
    $erreursTrouvees = $true
    continue
}

```

1. (**\$UtilisateurMatricule** -notmatch "**^[a-zA-Z0-9]+\$**") : Cela nous permet de vérifier que le matricule ne contient pas de caractères spéciaux et qu'il ne contient que des chiffres ou des lettres. Si ce n'est pas le cas l'utilisateur n'est pas créé.
2. (**\$UtilisateurMatricule** -notmatch "**^d{4,6}[a-zA-Z]{2,3}\$**") : Ce regex nous permet de vérifier le format du matricule, si il ne commence pas par 4 ou 6 chiffres, et ne finit pas par 2 ou 3 lettres, il ne sera pas créé.
3. (**\$UtilisateurBDD** -ne "" -and **\$UtilisateurBDD** -notmatch "**^BDD**") : Le début du champ BDD doit correspondre à « BDD »
4. (**\$UtilisateurMatricule.Substring(0,5)** -ne **\$UtilisateurAGRIH.Substring(0,5)**) : Les 5 premiers caractères du matricule doivent correspondre aux 5 premiers du matricule AGIRH sinon l'utilisateur est ignoré.

Quand toutes ces vérifications sont effectuées, tous les utilisateurs avec des champs incorrects seront écrit dans le fichier log et nous saurons ce qu'il y a à modifier.

Vérification dans l'Active Directory :

Groupe :

Une vérification est effectuée pour s'assurer que le groupe existe bien dans l'Active Directory. Si le groupe n'existe pas, une erreur est enregistrée et l'utilisateur est ignoré. Cette vérification se fait en cherchant le groupe dans l'AD à l'aide de son nom, et si aucun groupe correspondant n'est trouvé, une erreur est loguée et le script passe à l'utilisateur suivant.

```

if ($Groupe -and -not (Get-ADGroup -Identity $Groupe -ErrorAction SilentlyContinue)) {
    log "[ERROR] Ligne $index : Le groupe $Groupe n'existe pas dans l'AD."
}

```

```

$erreursTrouvees = $true
continue
}

```

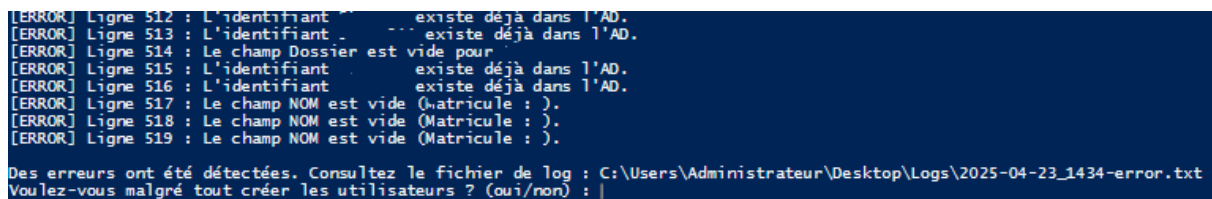
Résultat final :

Après que toutes les vérifications ont été effectuées, si le script a détecté des erreurs grâce au log et à la variable « \$erreursTrouvees ». Le script se met en pause et nous demande si malgré toutes les erreurs détectées nous voulions continuer à créer ou non les utilisateurs.

```

if ($erreursTrouvees) {
    Write-Host "`nDes erreurs ont été détectées. Consultez le fichier de log : $LogFile"
    $confirmation = Read-Host "Voulez-vous malgré tout créer les utilisateurs ? (oui/non)"
    if ($confirmation -eq "oui") {
        Write-Host "✅ Lancement du script de création des utilisateurs malgré les erreurs..."
    }
}

```



```

[ERROR] Ligne 512 : L'identifiant '...' existe déjà dans l'AD.
[ERROR] Ligne 513 : L'identifiant '...' existe déjà dans l'AD.
[ERROR] Ligne 514 : Le champ Dossier est vide pour '...'
[ERROR] Ligne 515 : L'identifiant '...' existe déjà dans l'AD.
[ERROR] Ligne 516 : L'identifiant '...' existe déjà dans l'AD.
[ERROR] Ligne 517 : Le champ NOM est vide (Matricule : ).
[ERROR] Ligne 518 : Le champ NOM est vide (Matricule : ).
[ERROR] Ligne 519 : Le champ NOM est vide (Matricule : ).

Des erreurs ont été détectées. Consultez le fichier de log : C:\Users\Administrateur\Desktop\Logs\2025-04-23_1434-error.txt
Voulez-vous malgré tout créer les utilisateurs ? (oui/non) : |

```

Exemple du script de vérification lancé

Nous pouvons voir que des erreurs ont été détectées pendant la vérification du fichier, on me demande donc si je veux toujours créer ou non les utilisateurs. Si je dis que je veux toujours créer les utilisateurs, le script de création sera lancé malgré les erreurs et les utilisateurs enregistrer avec des erreurs ne seront pas créés.²

² Le script complet sera affiché dans l'Annexe.

Script de désactivation et de suppression des utilisateurs

Après avoir terminé le script de création des utilisateurs, j'ai également réalisé deux scripts bien moins volumineux. Le premier script, le script de désactivation des utilisateurs servira quand un membre du personnel partira de l'hôpital. Ce script se référence également sur un fichier csv avec maintenant uniquement sur le matricule, le script fait une recherche des utilisateurs dans l'AD par rapport au matricule pour savoir s'il existe ou non. Quand cet utilisateur est trouvé, il sera déplacé dans l'OU A Detruire, dans un sou OU qui est l'année actuelle et dans l'année actuelle le mois actuel.

```
$moisNom = (Get-Date).ToString("MMMM", $cultureFR)
```

```
$annee = Get-Date -Format "yyyy"
```

```
$sousOU = "$mois - $($moisNom.ToUpper())"
```

```
$ouCible = "OU=$sousOU,OU=$annee,OU=A Detruire,DC=Stage,DC=net"
```

Ici le script récupère donc la date actuelle, surtout le mois et l'année, les mets dans une variable pour sélectionner l'OU. Si l'OU n'existe pas le script se coupera, s'il existe été que l'utilisateur existe, l'utilisateur sera désactivé, ensuite nous supprimons le mot de passe de son profil, nous l'enlevons des groupes, nous le déplaçons dans l'OU « A Detruire » et enfin nous activons le paramètre de la carte puce nécessaire pour la connexion.

```
Disable-ADAccount -Identity $matricule
```

```
log "Utilisateur $matricule desactive." -ForegroundColor Yellow
```

```
Set-ADUser -Identity $matricule -Clear telephoneNumber
```

```
Log "Champ telephoneNumber efface pour $matricule."
```

```
Remove-ADGroupMember -Identity "Tout le CHA" -Members $matricule -Confirm:$false
```

```
log "Utilisateur $matricule retire du groupe 'Tout le CHA'."
```

```
Move-ADObject -Identity $utilisateur.DistinguishedName -TargetPath $ouCible
```

```
log "Utilisateur $matricule deplace dans l'OU $sousOU." -ForegroundColor Green
```

```
Set-ADUser -Identity $Matricule -SmartcardLogonRequired $true
```

```
log "Option 'Carte a puce necessaire pour la connexion' activee pour $Matricule."
```

```

Utilisateur      desactive.
Champ telephoneNumber efface pour
Utilisateur      retire du groupe 'Tout le CHA'.
Utilisateur      deplace dans l'OU 05 - MAI.
Option 'Carte a puce necessaire pour la connexion' activee pour

```

Exemple d'un utilisateur désactivé grâce au script



Utilisateur dans l'OU

Ensuite j'ai créé un script de suppression des comptes, ce script supprime les utilisateurs présents dans un sous OU de A Detruire. Quand nous lançons le script, il nous demande quelle année et quel mois nous voulons supprimer.

```

PS C:\WINDOWS\system32> C:\Users\Administrateur\Desktop\Suppression.ps1
Entrez l'annee (ex: 2025) : 2025
Entrez le mois (ex: 05 pour mai) :

```

Lancement du script

Si la date que nous mettons ne correspond à aucun OU présent dans l'AD, il repose la question jusqu'à ce qu'on renseigne un OU existant. Ensuite quand nous mettons un OU existant, le script effectue une recherche dans tout cet OU, il recherchera si dans tous les utilisateurs il y a des étudiants, il les voit car dans leurs fonctions il y a écrit « Etudiant ». Si nous mettons « O » le script supprime les étudiants et nous mettons « N » le script gardera les étudiants dans l'AD. Nous faisons cela car un étudiant en stage peut partir et revenir dans 6 mois. Après avoir choisis le script se lancera, il supprimera uniquement les comptes désactivés et si le choix a été voulu les étudiants.

```

PS C:\WINDOWS\system32> C:\Users\Administrateur\Desktop\Suppression.ps1
Entrez l'annee (ex: 2025) : 2025
Entrez le mois (ex: 05 pour mai) : 05
Des utilisateurs sont identifiés comme etudiants. voulez-vous les supprimer ? (O/N) : |

```

Choix si oui ou non nous voulons supprimer les étudiants

```

Utilisateur      encore actif non supprime.
Utilisateur      encore actif non supprime.
⚠ Utilisateur      est un etudiant, mais non supprime (option choisie).
⚠ Utilisateur      est un etudiant, mais non supprime (option choisie).
⚠ Utilisateur      est un etudiant, mais non supprime (option choisie).
⚠ Utilisateur      est un etudiant, mais non supprime (option choisie).
⚠ Utilisateur      est un etudiant, mais non supprime (option choisie).
Utilisateur      supprime.
Utilisateur      supprime.

```

Résultat du script après avoir été exécuté (ici les étudiants ne seront pas supprimés)

Les utilisateurs actifs dans l'AD qui ne sont pas supprimés sont listés et leurs matricules sont envoyés dans un fichier texte, pour ensuite vérifier pourquoi ils sont actifs et possiblement les mettre dans leurs OU respectifs.

O - Fiona
C - Lucie
E - Theo

Utilisateur non supprimé avec matricule – Prénom NOM

Résumé et conclusion :

En résumé, ce projet a permis de développer une solution automatisée et fiable pour la création et la gestion des utilisateurs dans un environnement Active Directory. Grâce à un script PowerShell structuré, les processus auparavant manuels et sujets à ont été rationalisés. Cette automatisation répond à des exigences essentielles de sécurité, de traçabilité et de conformité, tout en apportant un gain de temps significatif pour les équipes informatiques.

En conclusion, le projet s'inscrit dans une démarche d'optimisation des services informatiques du groupe hospitalier, en améliorant la qualité des opérations, en réduisant les risques opérationnels et en garantissant une gestion centralisée et cohérente des identités numériques des collaborateurs.

II. Centralisation et supervision des Logs

Après avoir terminé mon script de création des utilisateurs avec leurs dossiers, leurs droits, leurs partages, et les vérifications, un autre projet m'a été confié. Ce projet rejoint en partie le projet précédent du Windows serveur.

J'ai eu comme projet d'utiliser un outil de centralisation de logs, les logs générés depuis le Windows serveur, le serveur de fichier et les clients. Le but est de traiter ces logs, de les envoyer vers ma centralisation de logs, les comprendre, les utiliser pour faire des graphiques et des alertes si besoin.

J'ai donc commencé par m'informer sur le fait de comment pousser des logs générés par le Windows serveur sur une machine Debian 12 qui devra les centraliser. Et un logiciel revenait énormément, c'est le logiciel Graylog accompagné de Winlogbeat, le logiciel Graylog est installé sur la machine Debian qui me servira à traiter ces logs et le logiciel Winlogbeat est un logiciel qui permet d'envoyer les logs à une adresse IP et un port.

Mise en place de Graylog et de Winlogbeat :

Pour mettre en place les deux services, je me suis aidé de deux sites internet qui expliquent comment mettre en place [Graylog](#) et [Winlogbeat](#). Après que les deux services ont été installés, je me suis renseigné pour commencer à pousser les logs sur Graylog. Pour faire cela il suffit seulement de modifier le fichier de configuration de Winlogbeat, dans le fichier de configuration il suffit de mettre l'IP de la machine Debian avec le port par défaut.

```
output.logstash:
```

```
hosts: ["192.168.137.132:5044"]
```

Voici la configuration pour envoyer tous les logs sur mon serveur Graylog. Mais j'aimerais pousser certains types de logs, par exemple les logs de création de compte, de suppression de compte, de connexion, de déconnexion...

Pour faire cela, nous avons qu'à prendre connaissance dans l'active directory l'ID des événements recherchés.

Dans l'observateur d'événement, nous pouvons aller dans l'onglet sécurité et voir les événements.³

Après avoir pris en compte les ID voulus j'ai vu que nous pouvions envoyer des ID spécifiques au serveur Graylog, j'ai donc décidé d'envoyer l'ouverture/ fermeture de session, la création/suppression de compte et les événements de sessions privilégiées.

³ Annexe (Visualisation ID AD)

Voici à quoi ressemble les ID dans le fichier :

```
winlogbeat.event_logs:
  - name: Application
    ignore_older: 72h
  - name: System
  - name: Security
    event_id: 4720, 4726, 4624, 4634, 4672
```

ID des événements choisis

Après avoir fait ces étapes de configuration de WinlogBeat, il faut maintenant configurer Graylog pour pouvoir recevoir les logs grâce à l'Input.

Pour faire cela dans l'interface de Graylog, aller dans le menu « [Système](#) » puis sur « [Inputs](#) ».

Quand nous sommes dans Inputs, sélectionner « [Beats](#) » puis « [Launch new input](#) ». Une interface va apparaître puis créer l'Input dont nous avons besoin. Quand elle sera créée un Input apparaîtra et nous verrons ceci :

Winlogbeat Input Beats (6800c162cc67b443d3d507b6) RUNNING
On node [★ 85d2babe / debian](#)

```
bind_address: 0.0.0.0
charset_name: UTF-8
no_beats_prefix: false
number_worker_threads: 2
override_source: <empty>
port: 5044
recv_buffer_size: 1048576
tcp_keepalive: false
tls_cert_file: <empty>
tls_client_auth: disabled
tls_client_auth_cert_file: <empty>
tls_enable: false
tls_key_file: admin
tls_key_password:*****
```

Input pour WinlogBeat

Voici la configuration de l'input pour recevoir les logs envoyer sur le port [5044](#) par le serveur Windows. Nous choisissons comme IP 0.0.0.0 pour que tout le monde puisse envoyer sur ce port, et le port 5044 car nous envoyons sur ce port quand nous avons configuré WinlogBeat.

Il nous suffit de lancer WinlogBeat dans le serveur Windows pour pouvoir pousser tous les logs.

Commande pour lancer winlog et pousser les infos.

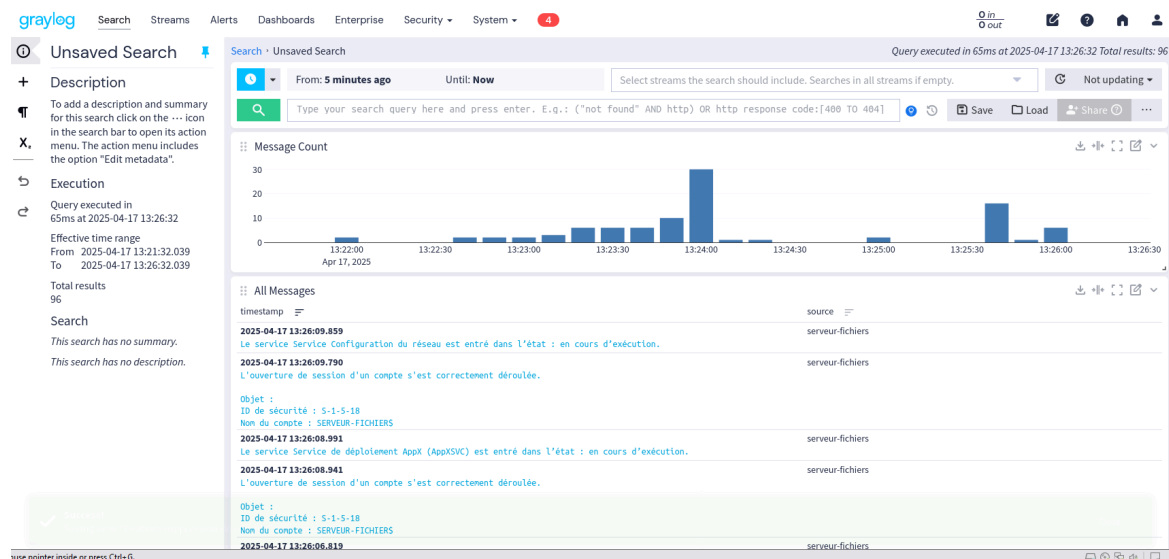
`./winlogbeat.exe -e`

```
{
  "log.level": "info",
  "@timestamp": "2025-04-17T15:11:55.883+0200",
  "log.logger": "monitoring",
  "log.origin": {
    "function": "github.com/elastic/beats/v7/libbeat/monitoring/report/log.(*reporter).logSnapshot",
    "file.name": "log/log.go",
    "file.line": 192
  },
  "message": "Non-zero metrics in the last 30s",
  "service.name": "winlogbeat",
  "monitoring": {
    "metrics": {
      "beat": {
        "cpu": {
          "system": {
            "ticks": 2921,
            "time": {
              "ms": 78
            }
          },
          "total": {
            "ticks": 4155,
            "time": {
              "ms": 94
            },
            "value": 4155,
            "user": {
              "ticks": 1234,
              "time": {
                "ms": 16
            }
          }
        },
        "info": {
          "ephemeral_id": "506d9604-baee-4dd7-94a4-3fe0de1cf2e8",
          "uptime": {
            "ms": 1230134
          },
          "version": "9.0.0"
        },
        "memstats": {
          "gc_next": 27987370,
          "memory_alloc": 24557808,
          "memory_total": 167298344,
          "rss": 64606208,
          "runtime": {
            "goroutines": 43
          }
        },
        "libbeat": {
          "config": {
            "module": "running": 0
          },
          "output": {
            "events": {
              "acked": 12,
              "active": 0,
              "batches": 3,
              "total": 12,
              "read": {
                "bytes": 18
              },
              "write": {
                "bytes": 7536,
                "latency": {
                  "histogram": {
                    "count": 93,
                    "max": 166,
                    "mean": 6.505376344086022,
                    "median": 1,
                    "min": 0,
                    "p75": 4,
                    "p95": 30.299999999999997,
                    "p99": 166,
                    "p999": 166,
                    "stddev": 18.917263922690587
                  }
                },
                "pipeline": {
                  "clients": 7,
                  "events": {
                    "active": 0,
                    "published": 10,
                    "total": 10,
                    "queue": {
                      "acked": 12,
                      "added": {
                        "events": 10
                      },
                      "consumed": {
                        "events": 12
                      },
                      "filled": {
                        "bytes": 0,
                        "events": 0,
                        "pct": 0
                      },
                      "max_bytes": 0,
                      "max_events": 3200,
                      "removed": {
                        "events": 12
                      }
                    },
                    "system": {
                      "handles": {
                        "open": 2
                      }
                    },
                    "ecs.version": "1.6.0"
                  }
                }
              }
            }
          }
        }
      }
    }
  }
}
```

Logs brut vu sur WinlogBeat

Maintenant que tout est configuré et lancé nous pouvons aller dans Graylog et visualiser les logs.

Nous voyons que tous types de logs sont envoyés depuis mon serveur Windows.



Ensemble des logs visualisés sur Graylog

Mais j'aimerais pouvoir visualiser des logs en particulier par exemple cibler un événement, donc pour pouvoir par exemple cibler l'événement de création d'utilisateurs.

Cibler ce genre d'événements permet de visualiser s'il y a des problèmes non voulus ou non prévus, par exemple si un administrateur n'est pas au courant qu'une création d'utilisateurs de masse est en cours il pourra savoir qui fait ça en direct.

Pour cibler un évènement je vais donc créer un « Stream » qui ciblera la création d'utilisateurs, pour faire cela je clique sur « Streams » dans le menu principal de Graylog. Ensuite « Create Stream », une fenêtre va apparaître, je lui donne le nom de « Stream de Création », puis « Create Stream ».

Create stream

Title

Stream de Création

A descriptive name of the new stream

Description (Opt.)

What kind of messages are routed into this stream?

Index Set

Default index set

Messages that match this stream will be written to the configured index set.

☐ Remove matches from 'Default Stream'

Don't assign messages that match this stream to the 'Default Stream'.

Cancel

Create stream

Création d'un Stream

Quand le Stream est créé je peux lui mettre des règles pour cibler mon évènement. Pour faire cela dans le Stream je clique sur « [More](#) » puis « [Quick Add Rule](#) ». Une fenêtre va s'ouvrir et nous pourrons cibler un évènement.

New Stream Rule

Field

winlogbeat_winlog_event_id

Type

match exactly

Value

4720

☐ Inverted

Description (Opt.)

Result: winlogbeat_winlog_event_id must match exactly 4720

Cancel

Create Rule

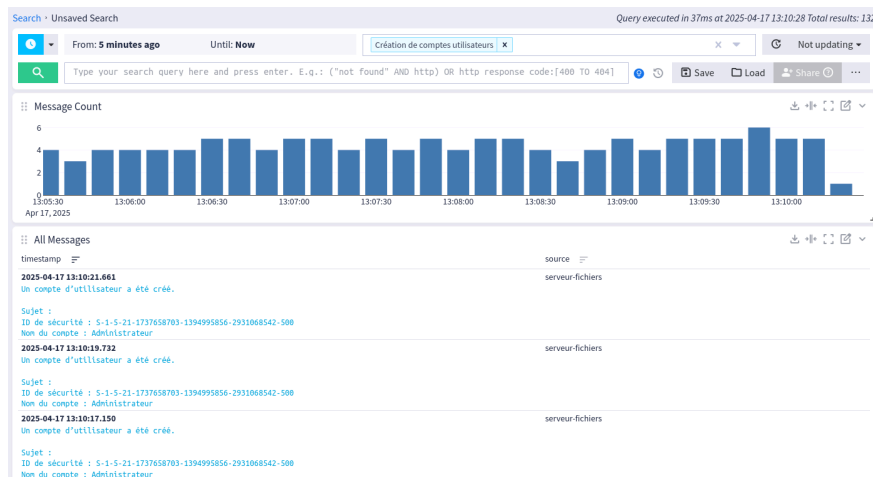
The server will try to convert to strings or numbers based on the matcher type as well as it can.
 [Take a look at the matcher code on GitHub](#)

Regular expressions use Java syntax.

Règle ciblage d'ID

Ici dans Field nous rentrons les évènements reçus depuis WinlogBeat, nous voulons cibler un évènement exact donc dans type nous mettons « [match exactly](#) » et dans value qui sera la valeur de l'évènement nous rentrons « [4720](#) » qui est l'évènement de création d'un utilisateur récupéré dans l'observateur d'évènement Windows. Nous n'avons plus qu'à créer la règle et lancer le Stream.

Maintenant que notre Stream est créé nous pouvons cliquer dessus pour le visualiser et une fenêtre s'ouvre devant nous avec un graphique.



Stream de création d'utilisateurs

Nous voyons maintenant uniquement les logs de création grâce au stream, nous voyons que c'est bien un stream car il y a un filtre « [Création de comptes utilisateurs](#) ».

Pour les autres événements à cibler la démarche est la même, donc je crée un stream pour visualiser la suppression des utilisateurs, la connexion et la déconnexion des utilisateurs et des personnes connectées avec des privilèges.

Après avoir créé les streams, je vais créer un tableau de bord avec des widgets pour pouvoir visualiser plusieurs choses en même temps. Je vais créer un tableau de bord pour visualiser les comptes créés et comptes supprimés.

Créer des tableaux de bord nous permet de visualiser, surveiller et analyser les logs de façon claire. Surtout car nous pouvons voir en temps réel les logs, nous pouvons voir les pics d'activité inhabituels, on peut aussi collaborer grâce à ce tableau de bord car il peut être partagé et d'autres choses.

Pour créer un tableau de bord, je vais dans le menu et je clique sur « [Dashboard](#) », je crée un Dashboard, dans ce dashboard⁴, je crée un widget. Ce widget sera pour afficher sous forme de courbe pour afficher la création des utilisateurs.

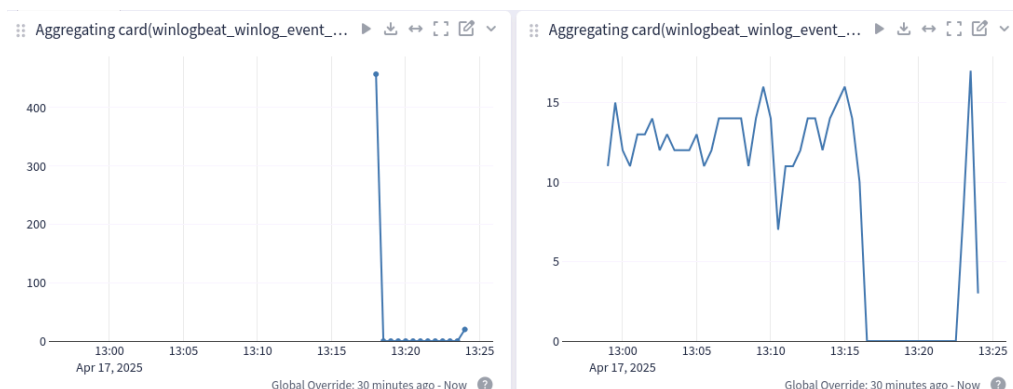
⁴ Tableau de bord

Création du widget de création d'utilisateurs

Dans ce [widget](#) nous affichons donc la création des utilisateurs en fonction du temps. Nous mettons un filtre au widget pour afficher uniquement les utilisateurs créés.

Je crée un deuxième widget dans le tableau de bord pour pouvoir afficher une courbe des utilisateurs supprimés.

Je place les widgets côte à côte pour pouvoir avoir un visuel groupé, ce qui est plus efficace si nous voulons étudier les courbes simultanément.



Voici le tableau de bord de la création et de la suppression des comptes.⁵

⁵ A droite nous avons la création des utilisateurs et à gauche la suppression.

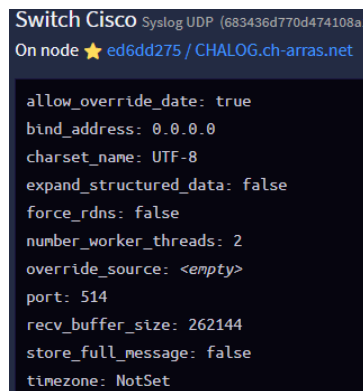
Graylog sur équipements réseaux:

Après la prise en main de graylog sur ma machine virtuelle, j'ai installé le service sur un serveur de l'hôpital pour recevoir et pouvoir traiter les logs des équipements informatiques.

Pour faire cela, allez sur votre équipement et définissez un serveur de logs. Pour faire cela faire les commandes suivantes :

```
SWITCH#conf t
SWITCH(config)#logging origin-id string SWITCH
SWITCH(config)#logging trap notifications
SWITCH(config)#logging host X.X.X transport udp port 514
SWITCH#copy running-config startup-config
```

Maintenant que le switch est configuré pour envoyer sur notre serveur graylog et sur le bon port, nous pouvons mettre en place notre INPUT graylog. Pour faire cela nous avons qu'à choisir Input de type Syslog UDP, et renseigner que nous recevons de toutes les adresses donc 0.0.0.0 sur le port 514.



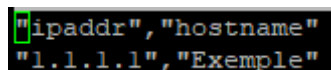
```
Switch Cisco Syslog UDP (683436d770d474108a1)
On node ★ ed6dd275 / CHALOG.ch-arras.net

allow_override_date: true
bind_address: 0.0.0.0
charset_name: UTF-8
expand_structured_data: false
force_rdns: false
number_worker_threads: 2
override_source: <empty>
port: 514
recv_buffer_size: 262144
store_full_message: false
timezone: NotSet
```

Input après le paramétrage

Maintenant nous allons dans la page Search et nous verrons toutes les logs de tous les équipements arriver, mais nous allons recevoir comme nom de sources des adresses IP et non des noms d'équipements, nous allons donc devoir configurer dans graylog un pipeline et une lookup table pour attribuer à chaque adresses IP un nom d'équipement.

Nous allons donc créer un fichier dans graylog au format csv, renseigner dans le nom de colonnes ipaddr et hostname.



```
"ipaddr", "hostname"
"1.1.1.1", "Exemple"
```

Exemple de la configuration du fichier

Après avoir fait cela se rendre dans graylog et dans l'onglet Lookup Tables puis dans [Data Adapters](#) où nous allons renseigner le chemin du fichier.

The screenshot shows the configuration for the 'IP_to_Hostname' data adapter, which is a CSV File. The configuration includes the following fields:

Field	Value
Description	Table de correspondance IP à nom d'hôte
File path	/etc/graylog/lookup/addr.csv
Separator	,
Quote character	"
Key column	ipaddr
Value column	hostname
Check interval	60 seconds
Case-insensitive lookup	no
CIDR lookup	no

An 'Edit' button is located at the bottom left of the configuration panel.

Configuration du data adapter

Ensuite je vais créer une règle dans un pipeline pour que quand une ip source est détecté dans graylog et qu'elle est dans le fichier elle sera renommée par le « [hostname](#) » équivalent à cette IP.

```
rule "resolve switch hostname from IP"
when
  has_field("source")
then
  let ip = to_string($message.source);
  let hostname = lookup_value("switch_lookup_table", ip);
  set_field("source", hostname);
end
```

Voici le code qui renomme la source automatiquement, nous pouvons faire une simulation.

The screenshot shows the 'Rule Simulation' interface. It has three tabs: 'JSON', 'Key Value', and 'Simple Message'. The 'JSON' tab is selected, showing a JSON object: `{ "source": "1.1.1.1" }`. Below the JSON input, there is a text field with the instruction: 'Enter a normal string to simulate the message field, Key-Value pairs or a JSON object'. There are two buttons: 'Run rule simulation' and 'Reset'. Below these buttons, a message ID is displayed: 'd4a5ba0b-8900-4713-af29-24c0ae6ea758' with a 'Not stored' status. At the bottom, there is a table with two columns: 'Timestamp' and 'source'. The 'source' column has the value 'Exemple'. Below the table, it says 'Stored in index' and 'Message is not stored'.

Simulation du changement de la source

Quand cela fonctionne nous pouvons aller voir dans la page de réception des logs pour s'assurer que toutes les sources sont bien renommées.

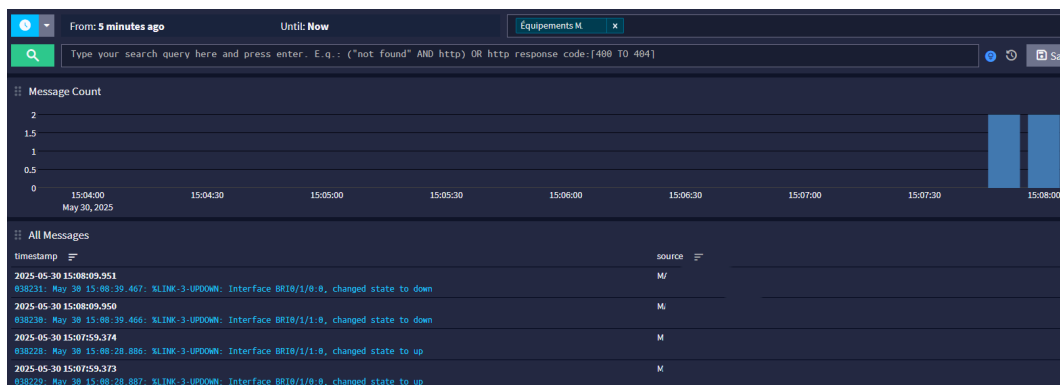
2025-05-30 13:52:46.047	-INFO
XLINKPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet3/0/19, changed state to up	
2025-05-30 13:52:45.047	-INFO-
XLINK-3-UPDOWN: Interface GigabitEthernet3/0/19, changed state to up	

Page de réception des logs

Après avoir vérifié que tous les équipements ont bien été renommés, nous pouvons faire des streams pour cibler les noms ou les endroits où ils se trouvent dans le Ternois ou l'Artois.



Streams créé selon l'endroit ou ce que le nom de l'équipement contient



Voici ce que nous voyons quand nous ciblons la recherche

Conclusion

Pour conclure, centraliser les logs Active Directory ou des équipements réseaux est une étape très importante dans des hôpitaux ou en entreprise, car cela permet de renforcer la sécurité en détectant rapidement les événements suspects et les tentatives d'intrusion, mais aussi de pouvoir tracer les événements critiques liés aux accès. La centralisation facilite les audits, et permet de réagir plus efficacement en cas de problèmes.

III. Création d'un serveur de gestion des logs CDR

Pourquoi faire cela ?

La téléphonie du GHAT est gérée avec un serveur Cisco, c'est généralement utilisé pour un environnement VoIP⁶, il gère généralement des services comme les appels, la messagerie vocale et les fonctionnalités de sécurité. Tous les appels effectués et gérés par ce serveur sont intégrés dans des fichiers au format CDR⁷, qui sont très compliqués à lire.

Le but du projet est de pouvoir intégrer les données des fichiers dans une base de données, et de pouvoir les traiter et les afficher sur une interface internet.

Format des fichiers

Les fichiers sont sous format « cdr » ressemblant à un fichier csv. Le fichier comporte 97 champs à traiter, il comporte les identifiants des appels, les IP utilisés, les numéros d'appelant et d'appelés, du temps, de la date et beaucoup d'autres informations.

Début du projet

Pour commencer je vais pouvoir utiliser un script, pour intégrer les données du fichier cdr dans la base de données, je me suis donc aidé d'une SAE à l'IUT de Béthune car le format des fichiers csv et cdr sont très similaires.

Je commence donc par installer sur une Machine Virtuelle Debian 12 les paquets dont j'aurais besoin, j'installe donc Python3, Flask, MariaDB, Nginx et d'autres.

Après avoir installé tous mes services, je vais donc commencer par créer mon script qui insérera toutes les données de mes fichiers cdr dans MariaDB. Je décide donc que le nom des colonnes sera fait par rapport à la première ligne du fichier cdr, et les données à insérer seront les données dans ces colonnes. Comme j'ai plusieurs fichiers cdr, je vais faire en sorte que si la colonne d'une donnée existe déjà, j'insère la donnée de la colonne du fichier csv dans la colonne de MariaDB existante.

Par exemple, comme dans tous mes fichiers CDR il y a la colonne « origIpAddr », si cette colonne n'existe pas dans la base de données, je la crée et j'insère les données dedans. Mais si cette colonne existe déjà, les données du fichier CDR de cette colonne seront intégrées à la table, de la base de données, existante.

⁶ Voice over IP

⁷ Call Data Record

Je commence donc à donner à mon script Flask la configuration de ma base de données :

```
DB_CONFIG = {  
    "host": "localhost",  
    "user": "cdr_utilisateur",  
    "password": " Progtr00",  
    "database": "cdr"  
}
```

J'ai créé un utilisateur qui aura les droits dans la base de données, il se nomme cdr_utilisateur et se connecte grâce à Progtr00 sur la base de données « cdr ».

Ensuite, je dis à mon script où il peut trouver les fichiers csv :

```
CDR_FOLDER = "/home/cdr"  
  
PROCESSED_FOLDER = "/home/traite"  
if not os.path.exists(PROCESSED_FOLDER):  
    os.makedirs(PROCESSED_FOLDER)
```

J'utilise ce dossier pour pouvoir envoyer tous les fichiers cdr que je traite dedans et ne pas mettre les mêmes données plusieurs fois dans la base de données. Puis je vérifie si le dossier existe, si le dossier n'existe pas je le crée.

```
COLUMNS_TO_KEEP = [  
    "dateTimeOrigination",  
    "callingPartyNumber",  
    "originalCalledPartyNumber",  
    "duration"  
]
```

Puis je mets dans une variable le nom des colonnes que je veux garder parmi les centaines présentes dans le fichier. Ces colonnes vont me donner la date, la durée, le numéro de l'appelant et de l'appelé qui seront importants dans l'affichage du site.

Comme la deuxième ligne de mon fichier [CDR](#) contient les types de données (comme FLOAT, VARCHAR, INT...), je l'utilise pour définir automatiquement les types des colonnes lors de la génération de la table SQL.

Par exemple, si une colonne a comme deuxième élément INT, je génère une instruction SQL où cette colonne est définie avec le type INT.

Ensuite j'utilise la fonction « [clean_table_name](#) » pour nettoyer le nom de la table, afin qu'il soit conforme, c'est-à-dire que les espaces, les tirets, les points virgules. Pour être certain que la table sera bien insérée et lisible.

```
def clean_table_name(name):  
    return name.strip().replace(" ", "_").replace(";", "").replace("-", "_")[:64]
```

Après avoir fait cela, je convertis les dates en dates lisibles (%Y-%m-%d %H:%M:%S) car nous l'avons en format brut et si nous voulons faire des recherches sur des dates précises nous ne pouvons pas. Dans cette fonction j'indique les dates de début de l'heure d'hiver jusqu'à la fin, je fais ça pour rajouter une heure quand nous sommes en heure d'été car UTC+1 et quand nous sommes en heure d'hiver je rajoute deux heures

```
def convert_timestamp(timestamp):  
    dt = datetime.datetime.fromtimestamp(int(timestamp))  
    start_date = datetime.datetime(2025, 3, 30)  
    end_date = datetime.datetime(2025, 10, 26)  
    if start_date <= dt <= end_date:  
        dt += timedelta(hours=2)  
    else:  
        dt += timedelta(hours=1)  
    return dt.strftime('%Y-%m-%d %H:%M:%S')
```

Ensuite pour créer les tables j'utilise la fonction « [ensure_table_exists](#) », qui me permet de créer les tables SQL à partir du nom de mes colonnes et de son type de données. Ensuite je réutilise la fonction « [clean_table_name](#) » pour rendre le nom de la table valide, puis vérifie si une table avec ce nom existe ou non, si la colonne n'existe pas et ne contient pas vidéo elle est créée. La fonction ajoute à chaque table deux colonnes id et value, et permet d'automatiser la création et l'organisation des tables.

```
def ensure_table_exists(col_name, col_type):  
    if col_name.lower() not in [col.lower() for col in COLUMNS_TO_KEEP]:  
        return None  
    table = clean_table_name(col_name)  
    cursor.execute(f"SHOW TABLES LIKE '{table}'")  
    if not cursor.fetchone():  
        if col_name.lower() in ["datetimeconnect", "datetimedisconnect", "datetimeorigination"]:  
            col_type = "VARCHAR(255)"  
        cursor.execute(f"""  
            CREATE TABLE `{table}` (  
                id INT AUTO_INCREMENT PRIMARY KEY,
```

```

        value {map_type(col_type)}

    );

    """

    conn.commit()

    print(f"✅ Table créée : {table} ({col_type})")

    return table

return table

```

La fonction `insert_value` permet d'insérer une valeur dans une table correspondant à une colonne spécifique, uniquement si cette colonne fait partie de la liste `COLUMNS_TO_KEEP`. Si la colonne correspond à un champ de type date (`dateTimeConnect`, `dateTimeDisconnect`, ou `dateTimeOrigination`), une conversion de la date est effectuée à l'aide de la fonction `convert_timestamp`. Ensuite, le nom de la table est nettoyé pour éviter les caractères indésirables, puis la valeur est insérée dans la colonne `value` de la table associée.

```

def insert_value(col_name, value):

    if col_name.lower() not in [col.lower() for col in COLUMNS_TO_KEEP]:

        return

    if col_name.lower() in ["datetimeconnect", "datetimedisconnect", "datetimeorigination"]:

        value = convert_timestamp(value)

    value = value.replace("'", "").strip()

    if value == "":

        value = ""

    table = clean_table_name(col_name)

    cursor.execute(f"INSERT INTO `{table}` (value) VALUES (%s);", (value,))

```

Pour finir je mets en place toutes les fonctions créer et j'exécute le script, il commence par parcourir les fichiers CDR et les lire, en extrayant les en-têtes de colonnes et les types de données depuis les premières lignes. Ensuite, pour chaque colonne, il vérifie si une table existe déjà, et si ce n'est pas le cas, elle est créée avec le bon type de données. Les données du fichier sont ensuite insérées dans la table correspondante, tout en vérifiant que chaque ligne contient bon nombre de colonnes. Ce processus permet d'automatiser l'importation et l'organisation des données provenant de fichiers CDR dans une base de données, tout en gérant les cas particuliers, comme les types de données ou les erreurs de format.

```

for filename in os.listdir(CDR_FOLDER):

    if filename.startswith("cdr") and "20" in filename:

        full_path = os.path.join(CDR_FOLDER, filename)

        print(f"\n Fichier : {filename}")

        with open(full_path, encoding='utf-8') as f:

            lines = [line.strip() for line in f if line.strip()]

```

```

if len(lines) < 3:
    print(" Trop peu de lignes, ignoré.")
    continue

headers = lines[0].split(',')
types = lines[1].split(',')
data_rows = lines[2:]

headers = [header.replace("'", "") for header in headers]

for i, col_name in enumerate(headers):
    if i >= len(types): continue
    table = ensure_table_exists(col_name, types[i])
    if table:
        for row in data_rows:
            values = row.split(',')
            if len(values) <= i:
                print(" Ligne ignorée (colonnes insuffisantes)")
                continue
            value = values[i].replace("'", "")
            insert_value(col_name, value)

conn.commit()

print(" Données insérées.")

```

Pour finir sur ce script, quand mon fichier est fini d’être traité je le renomme avec « t_ » devant le fichier puis je le déplace dans le dossier « /home/traite ».

```

new_filename = f"t_{filename}"
new_full_path = os.path.join(PROCESSED_FOLDER, new_filename)
shutil.move(full_path, new_full_path)
print(f" Fichier déplacé et renommé : {new_filename}")

```

```

MariaDB [cdr]> show tables;
+-----+
| Tables_in_cdr |
+-----+
| callingPartyNumber |
| dateTimeOrigination |
| duration |
| originalCalledPartyNumber |
| users |
+-----+
5 rows in set (0,000 sec)

MariaDB [cdr]> █

```

Exemple de tables insérées dans la base de données

Nous pouvons donc passer à la création de l'interface web. Pour cela, je vais utiliser Nginx pour gérer la partie HTTPS et rediriger le trafic vers cdr.com. Je vais utiliser les comptes rendus de SAE pour configurer cela, de plus je vais utiliser Flask pour la récupération des informations dans la base de données et je vais utiliser Gunicorn pour lancer l'application Flask, sur le port d'écoute de Nginx.

Je commence donc par créer mon Flask, dans ce fichier j'importe les modules dont j'aurais besoin, mysql.connector pour pouvoir me connecter sur la base de données et bcrypt pour pouvoir déchiffrer le mot de passe de l'utilisateur, qui pourra se connecter à la page, qui est renseigné dans la base de données.

Ensuite je renseigne, comme pour la première partie du projet, la configuration de la base de données pour pouvoir récupérer les informations.

```

def login_required(f):
    @wraps(f)
    def decorated_function(*args, **kwargs):
        if 'user_role' not in session or session['user_role'] != 'admin':
            return redirect(url_for('login'))
        return f(*args, **kwargs)
    return decorated_function

def hash_password(password):
    """Hash a password using bcrypt."""
    return bcrypt.hashpw(password.encode('utf-8'), bcrypt.gensalt())

def check_password(hashed_password, password):
    """Check a hashed password."""
    return bcrypt.checkpw(password.encode('utf-8'), hashed_password.encode('utf-8'))

```

Cette partie du script est utile pour la gestion de l'authentification et pour la sécurité des mots de passe dans l'application web. La fonction « login_required » permet de sécuriser l'accès à certaines routes de l'application, la fonction vérifier l'identifiant et si l'utilisateur a le rôle « admin », si le rôle n'est pas le bon l'utilisateur sera redirigé vers la page de connexion. Ensuite j'utilise la fonction bcrypt pour la sécurisation des mots de passe en les hachant, ce qui permet de protéger le mot de passe. Enfin, la fonction check_password est utilisée pour comparer un mot de passe en clair fourni par l'utilisateur avec un mot de passe haché stocké en base de données. Cette fonction assure que la vérification de l'authenticité des informations de connexion se fait de manière sécurisée, en utilisant une comparaison sûre entre les valeurs hachées. Ensemble, ces fonctions permettent de renforcer la sécurité des utilisateurs, en veillant à protéger les informations sensibles et en garantissant que seuls les utilisateurs authentifiés et autorisés puissent accéder à certaines parties de l'application.

La fonction [fetch_calls_by_datetime_range](#) permet de récupérer les appels effectués entre deux dates et heures données. Elle exécute une requête SQL sur la base de données pour extraire les informations des appels, comme la date, l'appelant, l'appelé et la durée. Si des numéros spécifiques d'appelants ou d'appelés sont fournis, ils sont utilisés comme filtres dans la requête. Les résultats seront utiles ensuite pour les afficher sur le site internet.⁸

La fonction « [fetch_id_by_calling_party_number](#) » me permet de récupérer l'ID qui correspond au numéro d'appelant donné dans « [callingPartyNumber](#) ». La fonction effectue une requête SQL pour rechercher l'ID associé au numéro dans la base de données. Donc si un numéro dans la base de données a pour ID 50, nous verrons ID = 50.⁹

La fonction « [fetch_all_data_for_id](#) » permet de récupérer toutes les données associées à un identifiant spécifique dans toutes les tables de la base de données. Elle commence par obtenir la liste de toutes les tables présentes dans la base, puis pour chaque table, elle effectue une requête SQL pour récupérer toutes les lignes où l'ID correspond à l'ID fourni.¹⁰

Pour finir le script, trois pages html sont créées :

Page d'accueil (/)

Cette page est la page de base quand nous arrivons sur le site, elle est automatiquement redirigée sur la page /login.

Page de connexion (/login)

Cette page est la page qui permet de se connecter au site, quand je suis connecté je suis redirigé vers /search. Quand la méthode est GET, la page affiche le formulaire de connexion. Lorsque l'utilisateur essaye de se connecter, il soumet le formulaire avec la méthode POST, et les informations sont vérifiées dans la base de données.

⁸ [Fonction dans l'annexe](#)

⁹ [Script fetch_id](#)

¹⁰ [Script fetch_all_data_id](#)

Page de recherche (/search)

Cette page est uniquement accessible lorsque l'utilisateur a réussi à se connecter. Sur cette page nous pouvons faire des recherches dans la base de données et vérifier des informations si nous en avons besoin. Il peut chercher un numéro de téléphone et affiner la recherche grâce aux dates que nous récupérerons dans la base de données, cela nous donnera tous les appels entre deux dates. Enfin les résultats sont affichés sur plusieurs pages si trop conséquents, il y aura 20 lignes par page.

Page de recherche par appelant (/search_caller)

Cette page permet à l'utilisateur de voir toutes les informations sur un utilisateur grâce à l'ID que nous avons récupéré grâce à « fetch_all_data_id ».

Page de statistique (/stats)

Cette page me permet d'envoyer à la page stats.html, les informations sur le nombre d'appels, la durée et de savoir le nombre d'appels entrant et sortant. Cela compte le nombre d'appels par jour, je fais une moyenne de la durée par rapport aux appels et de compter le nombre de caractères par numéros, si le numéro de l'appelant comporte plus de 5 chiffres c'est un appel entrant et il comporte 5 chiffres c'est un appel sortant.

Ensuite je crée les pages html simples, ce qui permettra d'avoir des pages internet très simples à comprendre.

Page login.html :

Lorsque l'utilisateur soumet son nom d'utilisateur et son mot de passe, Flask se connecte à la base de données MySQL pour vérifier les informations. Si les informations sont valides, le rôle de l'utilisateur est stocké dans la session et il est redirigé vers la page protégée, sinon un message d'erreur est affiché.

Page search.html :

Le formulaire HTML permet de chercher des appels en fonction de deux critères : une plage de dates et heures, ou simplement le numéro de l'appelant. Lorsque l'utilisateur soumet le formulaire, Flask prend ces informations, effectue une recherche dans la base de données et affiche les résultats qui correspondent aux critères choisis.

Page results.html :

Cette page me permet d'afficher les informations que nous avons demandées dans la page /search. J'affiche donc sous forme de tableau toutes les informations, chaque ligne correspond à un appel passé dans cette plage de dates. J'ai fait en sorte de pouvoir avoir plusieurs pages et d'afficher 20 appels par page (pagination). J'ai aussi ajouté deux boutons, un bouton pour télécharger le contenu de la page actuelle et un autre pour télécharger le résultat de toutes les pages.

Page stats.html :

Cette page html me permet d'afficher un graphique sur le nombre d'appels dans une plage de dates données, d'afficher la durée moyenne des appels, le nombre d'appels entrant et sortant. Je peux aussi me rediriger vers la page /results en gardant les infos voulues pour afficher en détails ce que nous voyons dans le graphique.

Quand toutes les pages sont faites, il ne reste plus qu'à faire la configuration du fichier Nginx et puis nous allons pouvoir lancer le site.

Le site web sera lancé à l'aide de Gunicorn, en écoutant sur l'adresse 0.0.0.0:8000. Cela signifie que Gunicorn accepte les connexions provenant de n'importe quelle interface réseau de la machine. Nginx redirige toutes les requêtes vers l'application Flask.

```
location / {  
    proxy_pass http://127.0.0.1:8000;  
    proxy_set_header Host $host;  
    proxy_set_header X-Real-IP $remote_addr;  
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
    proxy_set_header X-Forwarded-Proto $scheme;  
}
```

Pour finir j'accède sur le site depuis mon DNS (cdr.ch-arras.net), qui a été configuré pour pointer vers l'IP publique de mon serveur.

Quand tout à été fait, je peux donc lancer l'application flask sur le port 8000 grâce à gunicorn.

```
gunicorn --bind 0.0.0.0:8000 app:app
```

Nous pouvons maintenant aller sur le site :



Page d'accueil du site internet.

Maintenant nous pouvons faire nos recherches, quand l'utilisateur admin se connecte, il atterrit sur la page de recherche dans la base de données.

Recherche par plage de dates et heures

Impossible de récupérer la plage de dates disponible.

Date et heure de début :

Date et heure de fin :

Nombres de l'appelant (séparés par des virgules, optionnel) :

Nombres de l'appelé (séparés par des virgules, optionnel) :

[Rechercher](#)

[Voir les statistiques](#)

Page de recherche

Sur cette page vous pouvez choisir entre quelle date et quelle date vous voulez voir vos appels, si vous mettez entre le 01 Mai 2025 et le 05 Mai 2025 vous verrez tous les appels effectués entre ces deux dates.

Résultats pour les appels entre 2025-05-01 10:10:00 et 2025-05-05 10:17:00

Appels trouvés :

Date	Appelant	Appelé	Durée (secondes)
2025-05-02 06:07:39	57		0
2025-05-02 06:07:45	53		0
2025-05-02 10:56:36	7	006	273
2025-05-02 11:00:53	4	0077	4
2025-05-02 11:00:49	0067	4	5
2025-05-02 11:00:18	337	33	68
2025-05-02 10:59:52	43	006	86
2025-05-02 11:00:36	41	41	57
2025-05-02 11:01:31	86	850	0
2025-05-02 11:01:35	4	0032	0
2025-05-02 11:01:18	97	4	10

Exemple de résultat

Les données sont maintenant affichées et nous pouvons voir qui a effectué des appels entre ces deux dates.

Vous pouvez aussi affiner la recherche sur un numéro en particulier.

Résultats pour les appels entre 2025-04-30 10:23:00 et 2025-05-05 10:23:00

Appels trouvés :

Numéros de l'appelant recherchés : 33

Date	Appelant	Appelé	Durée (secondes)
2025-05-02 11:00:15	33.		68
2025-05-02 13:49:30	33.	31	22
2025-05-02 09:06:04	33	33	289

Recherche sur un numéro en particulier

Je peux aussi faire la recherche sur qui a appelé une personne en particulier, ou je peux chercher plusieurs appelant en séparant les deux numéros d'une virgule.

[Télécharger cette page](#)

[Télécharger toutes les pages](#)

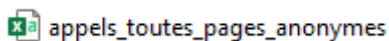
Page 1 sur 24370 [Suivant](#) [Dernière](#)

Boutons pour télécharger le contenu de la page de résultats



Téléchargement d'une page

J'ai aussi effectué un système d'anonymat, c'est-à-dire que si nous voulons télécharger la liste des appels, tous les numéros qui sont extérieurs à l'hôpital donc 06, 07 etc, seront anonymisés avec les 4 derniers chiffres remplacés par un X.



Fichier téléchargé contenant les appels anonymes

2025-05-03 09:38:08,068809XXXX,

Numéro anonyme

Une page de statistiques avec des graphiques est aussi disponible sur le site, nous pouvons y voir la durée des appels en moyenne, le nombre d'appels par jour, les appels entrants, sortants. Si nous choisissons de filtrer avec un numéro il y aura une ligne supplémentaire indiquant combien de fois il a été appelé et combien de fois il a appelé.

Statistiques d'Appels

Date et Heure de Début : 01/05/2025 09:39

Date et Heure de Fin : 03/05/2025 09:39

Filter par Numéro (optionnel) : Ex: 12345

Afficher les statistiques

Voir les résultats

Statistiques du 2025-05-01 09:39:00 au 2025-05-03 09:39:00

Total d'appels : 25519

Durée moyenne des appels : 45.81 secondes

Appels entrants(numéro de plus de 5 chiffres) : 4729

Appels sortants (numéro de 5 chiffres ou moins) : 20790

Heure de pointe : 2025-05-02 avec 18384 appels

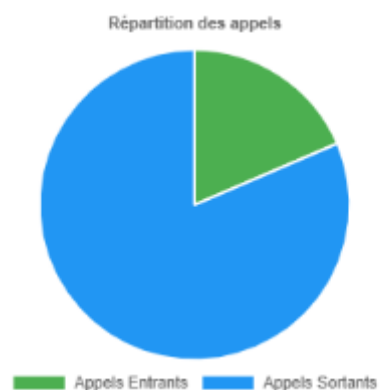
Page de statistiques sur plusieurs jours



Page de statistiques sur une journée

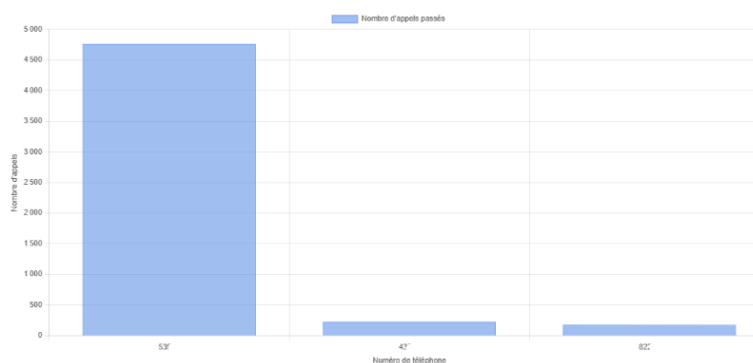
Le graphique s'adapte en fonction de la période choisie : s'il couvre plusieurs jours, il affichera les données par jour, et s'il est limité à une seule journée, il affichera les données par heure.

J'ai aussi un graphique affichant un camembert sur la répartition des appels entrant (numéro de plus de 5 chiffres) et sortant (numéro de 5 chiffres)



Graphique en camembert sur la répartition des appels

Et un dernier graphique représentant les 3 numéros passant le plus d'appels dans l'hôpital.



Graphique des 3 numéros passant le plus d'appels

La plupart du temps ces numéros affichés représentent des ascenseurs, des badgeuses etc..

Conclusion :

Ce projet a tout d'abord été réalisé sur environnement virtuel et adapté pour le serveur physique de l'hôpital, ça m'a permis d'apprendre davantage sur la gestion de la base de données et de flask, j'ai aussi appris à lire et à comprendre les fichiers cdr. Ce serveur sera utilisé au sein de l'hôpital pour avoir des informations s'il y a un problème dans la gestion des appels.

Conclusion

Ce stage m'a permis de consolider les compétences acquises durant mes deux années de BUT Réseaux et Télécommunications tout en en développant de nouvelles, directement en lien avec les missions du métier d'administrateur systèmes et réseaux. J'ai pu manipuler des équipements réels, configurer des serveurs, créer des scripts avancés en PowerShell, superviser les logs d'infrastructure via Graylog et mettre en œuvre des projets concrets comme la centralisation des logs ou encore la création d'un service web complet de gestion de fichiers CDR. Ces expériences pratiques m'ont offert une vision plus technique du métier, tout en me confortant dans mes choix d'orientation.

Sur le plan humain, ce stage a été particulièrement enrichissant. Intégré au sein du service informatique du GHAT, j'ai pu collaborer avec des professionnels expérimentés et passionnés, qui m'ont transmis leurs méthodes de travail et leurs conseils. Travailler en équipe, savoir s'adapter aux contraintes du terrain et communiquer efficacement avec différents corps de métier au sein de l'hôpital m'a permis de gagner en assurance, en autonomie, mais également en rigueur.

Enfin, cette immersion dans le monde professionnel a eu un impact direct sur ma vision du métier et de mon avenir. Ce stage a renforcé mon envie de poursuivre dans le domaine de l'administration systèmes et réseaux, un domaine à la fois technique et stimulant, en constante évolution. Je souhaite donc approfondir mes compétences dans ce secteur en poursuivant mes études en alternance dans une licence professionnelle ou en effectuant un master en cybersécurité ou administration réseau. Ce stage m'a conforté dans cette voie, en me montrant à quel point ce métier peut être motivant et porteur de responsabilités concrètes.

Outils utilisés lors du stage

Je vais présenter et expliquer les outils que j'ai utilisés pendant ce stage.

KeePass : Cet outil est utilisé pour générer et stocker des mots de passes fort, c'est un outil que j'ai utilisé tout au long de mon stage.

Putty : J'ai utilisé ce logiciel pour pouvoir me connecter en SSH quand j'avais besoin sur le serveur CDR.

WinSCP : Ce logiciel m'a servi à téléverser des documents, fichier, images etc. Sur le serveur CDR où je ne pouvais rien télécharger. J'avais seulement à aller sur ce logiciel puis téléverser de ma machine physique au serveur CDR les documents.

VMware Workstation Pro : Avec VMware j'ai pu me créer un espace virtuel, et réaliser mes projets en local sans impacter de réseau de l'hôpital. C'est très similaire à VirtualBox.

Source :

[PowerShell : exécuter des commandes et scripts à distance](#)

[Centralisation des logs : installation de Graylog sur Debian](#)

[Installer Winlogbeat et Packetbeat sur Windows: Un tutoriel complet](#)

[Call Detail Records Administration Guide for Cisco Unified Communications Manager, Release 11.5\(1\) - Cisco Call Management Record Field Descriptions \[Cisco Unified Communications Manager \(CallManager\)\] - Cisco](#)

[Le Chat - Mistral AI](#)

[How to get started with MySQL and MariaDB](#)

[python - Convert to UTC Timestamp - Stack Overflow](#)

[How to Create a Flask API with MariaDB on AWS EC2 | GeeksforGeeks](#)

[Setting Up MariaDB - Smile CDR Documentation](#)

[Chart.js](#)

[Docker Guide | Stirling-PDF](#)

[Stirling-PDF/src/main/resources/templates at main · Stirling-Tools/Stirling-PDF · GitHub](#)

[Usage Monitoring | Stirling-PDF](#)

<https://github.com/rconfig>

<https://docs.rconfig.com/tasks/scheduled-tasks/>

Annexe :

Utilisateurs dans l'Active Directory :

Exemple dans le fichier CSV :

NOM;Prenom;Matricule1;MatriculeAGRIH;Bureau;Fonction;OU;Path;Dossier;Grade;BDD;;;DateEXP

DUVAL;Baptiste;12345BD;123456;Stagiaire;Stagiaire;OU=Users,DC=Stage,DC=net;E:\Users;Interne;(Int);BDD9;;;02/07/2026

The screenshot shows the 'Propriétés de : DUVAL Baptiste' window with the 'Général' tab selected. The window contains the following fields:

- Prénom : Baptiste
- Initiales : (empty)
- Nom : DUVAL
- Nom complet : DUVAL Baptiste
- Description : (empty)
- Bureau : Stagiaire
- Numéro de téléphone : (empty)
- Adresse de messagerie : (empty)
- Page Web : (empty)

Buttons at the bottom: OK, Annuler, Appliquer, Aide.

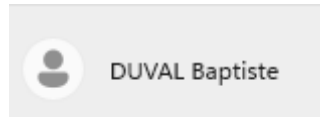
Profil général de l'utilisateur

The screenshot shows the 'Propriétés de : DUVAL Baptiste' window with the 'Profil' tab selected. The window contains the following fields and options:

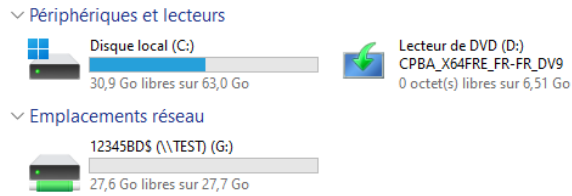
- Nom d'ouverture de session de l'utilisateur : Baptiste.DUVAL @Stage.net
- Nom d'ouverture de session de l'utilisateur (antérieur à Windows 2000) : STAGE\ 12345BD
- Horaires d'accès... (button)
- Se connecter à... (button)
- ☒ Déverrouiller le compte
- Options de compte :
 - ☒ L'utilisateur devra changer le mot de passe
 - ☐ L'utilisateur ne peut pas changer de mot de passe
 - ☐ Le mot de passe n'expire jamais
 - ☐ Enregistrer le mot de passe en utilisant un chiffrement réversible
- Date d'expiration du compte:
 - ☐ Jamais
 - ☒ Fin de : mercredi 1 juillet 2026

Buttons at the bottom: OK, Annuler, Appliquer, Aide.

Profil de l'utilisateur



Utilisateur connecté



Disque de l'utilisateur mappé

SCRIPT DE CRÉATION :

Variables de configuration

```
$formattedDate = Get-Date -Format "yyyy-MM-dd_HH:mm"
```

```
$LogFile = "C:\Scripts\Logs\$formattedDate-error.txt"
```

```
$CSVFile = "C:\Scripts\int.csv"
```

```
$cheminFichier = "C:\Scripts\Commandes-CHAUSERS-$formattedDate.txt"
```

```
$cheminMail = "C:\Scripts\Commandes-CHAEXCH-$formattedDate.txt"
```

```
$controle = "C:\Scripts\Controle-$formattedDate.txt"
```

Importation des données du fichier CSV

```
$CSVData = Import-Csv -Path $CSVFile -Delimiter ";" -Encoding UTF8
```

Groupes définis

```
$Groupemdp = "Strategie Mot de Passe"
```

```
$Groupesillage = "Sillage"
```

```
$Groupecha = "Tout le CHA"
```

```
$Groupemedecin = "Medecins CHA"
```

```
$GroupeInternesCHA = "Internes CHA"
```

Vérification si le fichier de Info existe, sinon création

```
if (-Not (Test-Path $LogFile)) {  
    New-Item -Path $LogFile -ItemType File  
}
```

Fonction pour écrire un message d'erreur en rouge dans le log

```

function Error {
    param ([string]$message)
    Write-Host $message -ForegroundColor Red
    Add-Content -Path $LogFile -Value "[ERROR] $message"
}

# Fonction pour ecrire un message d'avertissement en jaune dans le log
function Warning {
    param ([string]$message)
    Write-Host $message -ForegroundColor Yellow
    Add-Content -Path $LogFile -Value "[WARNING] $message"
}

# Fonction pour ecrire un message d'information en vert dans le log
function Info {
    param ([string]$message)
    Write-Host $message -ForegroundColor Green
    Add-Content -Path $LogFile -Value "[INFO] $message"
}

# Fonction pour ajouter des messages dans un fichier specifique
function fichier {
    param ([string]$message)
    Add-Content -Path $cheminFichier -Value $message
}

# Fonction pour ajouter des commandes de mail dans un fichier
function mail {
    param ([string]$message)
    Add-Content -Path $cheminMail -Value $message
}

# Fonction pour ajouter des contrôles dans un fichier specifique
function controle {
    param ([string]$message)
    Add-Content -Path $controle -Value $message
}

```

```

}

# Variable pour suivre les erreurs
$erreursTrouvees = $false

$index = 0

# Parcourir chaque utilisateur du fichier
foreach ($Utilisateur in $CSVData) {
    $index++

    $UtilisateurNom = $Utilisateur.NOM
    $UtilisateurPrenom = $Utilisateur.Prenom
    $UtilisateurMatricule = $Utilisateur.Matricule
    $UtilisateurAGRIH = $Utilisateur.EmployeeID
    $UtilisateurBureau = $Utilisateur.Service
    $UtilisateurFonction = $Utilisateur.Fonction
    $UtilisateurOU = $Utilisateur.OU
    $UtilisateurPath = $Utilisateur.Path
    $UtilisateurDossier = $Utilisateur.Dossier
    $UtilisateurBDD = $Utilisateur.ExchBDD
    $UtilisateurPN = "$UtilisateurPrenom.$UtilisateurNom@Stage.fr"
    $UtilisateurDate = $Utilisateur.DateEXP
    $UtilisateurGrade = $Utilisateur.Grade
    $Groupe = $Utilisateur.Groupe
    $Sillage = $Utilisateur.Sillage

    #Verification des erreurs sans pour autant passer la ligne
    if ([string]::IsNullOrEmpty($UtilisateurNom)) {
        Error "Ligne $index : Le champ NOM est vide (Matricule : $UtilisateurMatricule)."
        $erreursTrouvees = $true
    }

    if ([string]::IsNullOrEmpty($UtilisateurPrenom)) {
        Error "Ligne $index : Le champ PRENOM est vide pour $UtilisateurNom."
        $erreursTrouvees = $true
    }
}

```

```

if ([string]::IsNullOrEmpty($UtilisateurMatricule)) {
    Error "Ligne $index : Le champ Matricule est vide pour $UtilisateurNom $UtilisateurPrenom."
    $erreursTrouvees = $true
}

if ([string]::IsNullOrEmpty($UtilisateurAGRIH)) {
    Error "Ligne $index : Le champ AGRIH est vide pour $UtilisateurNom $UtilisateurPrenom."
    $erreursTrouvees = $true
    continue
}

if ([string]::IsNullOrEmpty($UtilisateurDossier)) {
    Warning "Ligne $index : Le champ Dossier est vide pour $UtilisateurNom $UtilisateurPrenom."
    $erreursTrouvees = $true
}

if ([string]::IsNullOrEmpty($UtilisateurDate)) {
    Error "Ligne $index : Le champ Date est vide pour $UtilisateurNom."
    $erreursTrouvees = $true
}

if(Get-ADUser -Filter {SamAccountName -eq $UtilisateurMatricule} -Properties DistinguishedName){
    Warning "Ligne $index : L'utilisateur existe dans l'AD"
    $erreursTrouvees = $true
}

# Verification de format
if ($UtilisateurMatricule -notmatch "^[a-zA-Z0-9]+$") {
    Error "Ligne $index : Le matricule $UtilisateurMatricule contient des caracteres non autorises."
    $erreursTrouvees = $true
}

if ($UtilisateurMatricule -notmatch "^\d{4,6}[a-zA-Z]{2,3}$") {
    Warning "Ligne $index : Le matricule $UtilisateurMatricule ne respecte pas le format attendu."
}

```

```

}

if ($UtilisateurBDD -ne "" -and $UtilisateurBDD -notmatch "^BDD") {
    Error " Ligne $index : Le champ BDD '$UtilisateurBDD' ne commence pas par 'BDD'."
    $erreursTrouvees = $true
}

if ($UtilisateurMatricule.Substring(0,5) -ne $UtilisateurAGRIH.Substring(0,5)) {
    Error " Ligne $index : Les 5 premiers caracteres du matricule '$UtilisateurMatricule' ne correspondent pas a ceux de '$UtilisateurAGRIH'."
    $erreursTrouvees = $true
}

# Verification dans l'AD
if (Get-ADUser -Filter {SamAccountName -eq $UtilisateurMatricule}) {
    Error " Ligne $index : L'identifiant $UtilisateurMatricule existe deja dans l'AD."
    $erreursTrouvees = $true
}

if (Get-ADUser -Filter {SamAccountName -eq $UtilisateurAGRIH}) {
    Error " Ligne $index : L'identifiant AGRIH $UtilisateurAGRIH existe deja dans l'AD."
    $erreursTrouvees = $true
}

if (Get-ADUser -Filter {UserPrincipalName -eq $UtilisateurPN}) {
    Error " Ligne $index : L'adresse UPN $UtilisateurPN existe deja dans l'AD."
    $erreursTrouvees = $true
}

# Verification de l'existence du groupe
if ($Groupe -and -not (Get-ADGroup -Identity $Groupe -ErrorAction SilentlyContinue)) {
    Error " Ligne $index : Le groupe $Groupe n'existe pas dans l'AD."
    $erreursTrouvees = $true
}
}

$confirmation = "non"

```

```

if ($ErreursTrouvees) {
    $confirmation = Read-Host "Des erreurs ont ete detectees. Voulez-vous malgre tout creer les utilisateurs ? (oui/non)"
}

if (-not $ErreursTrouvees -or $confirmation -eq "oui") {
    Write-Host "✅ Lancement du script de creation des utilisateurs..."

    $MatriculesTraites = @()

    foreach ($Utilisateur in $CSVData) {
        $index++

        # Recuperation des informations de l'utilisateur
        $UtilisateurNom = $Utilisateur.NOM
        $UtilisateurPrenom = $Utilisateur.Prenom
        $UtilisateurMatricule = $Utilisateur.Matricule
        $UtilisateurAGRIH = $Utilisateur.EmployeeID
        $UtilisateurBureau = $Utilisateur.Service
        $UtilisateurFonction = $Utilisateur.Fonction
        $UtilisateurOU = $Utilisateur.OU
        $UtilisateurPath = $Utilisateur.Path
        $UtilisateurDossier = $Utilisateur.Dossier
        $UtilisateurBDD = $Utilisateur.ExchBDD
        $UtilisateurDate = $Utilisateur.DateEXP
        $UtilisateurGrade = $Utilisateur.Grade
        $Groupe = $Utilisateur.Groupe
        $Sillage = $Utilisateur.Sillage

        # Suppression des espaces dans le NOM et PRENOM
        $UtilisateurNom = $UtilisateurNom -replace '\s', ' '
        $UtilisateurPrenom = $UtilisateurPrenom -replace '\s', ' '

        # Mise en forme du prenom (premiere lettre en majuscule et le reste en minuscule)
        $UtilisateurPrenom = $UtilisateurPrenom.Substring(0,1).ToUpper() + $UtilisateurPrenom.Substring(1).ToLower()

        # Creation de l'adresse email de l'utilisateur
        $UtilisateurPN = "$UtilisateurPrenom.$UtilisateurNom@Stage.fr"
    }
}

```

```

# Verification des champs obligatoires

if ([string]::IsNullOrEmpty($UtilisateurNom)) {
    Error " Ligne $index : Le champ NOM est vide (Matricule : $UtilisateurMatricule)."
    $erreursTrouvees = $true
    continue
}

if ([string]::IsNullOrEmpty($UtilisateurPrenom)) {
    Error " Ligne $index : Le champ PRENOM est vide pour $UtilisateurNom."
    $erreursTrouvees = $true
    continue
}

if ([string]::IsNullOrEmpty($UtilisateurMatricule)) {
    Error " Ligne $index : Le champ Matricule est vide pour $UtilisateurNom $UtilisateurPrenom."
    $erreursTrouvees = $true
    continue
}

if ([string]::IsNullOrEmpty($UtilisateurAGRIH)) {
    Error " Ligne $index : Le champ AGRIH est vide pour $UtilisateurNom $UtilisateurPrenom."
    $erreursTrouvees = $true
    continue
}

if ([string]::IsNullOrEmpty($UtilisateurOU)) {
    Error " Ligne $index : Le champ OU est vide pour $UtilisateurNom $UtilisateurPrenom."
    $erreursTrouvees = $true
    continue
}

```

```

# Verification de format du matricule

if ($UtilisateurMatricule -notmatch "[a-zA-Z0-9]+$") {

    Error " Ligne $index : Le matricule $UtilisateurMatricule contient des caracteres non autorises."

    $erreursTrouvees = $true

    continue

}

if ($UtilisateurBDD -ne "" -and $UtilisateurBDD -notmatch "^BDD") {

    Error " Ligne $index : Le champ BDD '$UtilisateurBDD' ne commence pas par 'BDD'."

    $erreursTrouvees = $true

    continue

}

# Verification des 5 premiers caracteres du matricule par rapport a AGRIH

if ($UtilisateurMatricule.Substring(0, 5) -ne $UtilisateurAGRIH.Substring(0, 5)) {

    Error " Ligne $index : Les 5 premiers caracteres du matricule '$UtilisateurMatricule' ne correspondent pas a ceux de '$UtilisateurAGRIH'."

    $erreursTrouvees = $true

}

# Recherche de l'utilisateur dans l'AD

$userMatriculeInAD = Get-ADUser -Filter "SamAccountName -eq '$UtilisateurMatricule'" -Properties DistinguishedName |
Where-Object { $_.DistinguishedName -notlike "*OU=A_supprimer,*" }

$userInSupprimer = Get-ADUser -Filter "SamAccountName -eq '$UtilisateurMatricule'" -Properties DistinguishedName |
Where-Object { $_.DistinguishedName -like "*OU=A_supprimer,*" }

# Si l'utilisateur existe deja dans l'AD et n'est pas dans l'OU "A_supprimer"

if ($userMatriculeInAD) {

    $userMatriculeOU = $userMatriculeInAD.DistinguishedName

    try {

        # Mise a jour de l'utilisateur

        Set-ADUser -Identity $UtilisateurMatricule `

            -UserPrincipalName "$UtilisateurPN" `

```

```

-Title "$UtilisateurFonction" `
-Department "$UtilisateurBureau" `
-Office "$UtilisateurBureau" `
-EmployeeID "$UtilisateurAGRIH" `
-AccountExpirationDate "$UtilisateurDate"

# Deplacement de l'utilisateur dans la bonne unite organisationnelle (OU)
if ($UserMatriculeOU -ne $UtilisateurOU) {
    Move-ADObject -Identity $UserMatriculeOU -TargetPath $UtilisateurOU
}

Info "L'utilisateur $UtilisateurMatricule a ete mis a jour avec succes."
$MatriculesTraites += $UtilisateurMatricule
} catch {
    Error " Impossible de mettre a jour l'utilisateur $UtilisateurMatricule : $_"
    $erreursTrouvees = $true
    continue
}
}

# Si l'utilisateur existe dans l'OU "A_supprimer", reactivation et mise a jour
elseif ($UserInSupprimer) {
    try {
        # Deplacement de l'utilisateur hors de l'OU "A_supprimer"
        Move-ADObject -Identity $UserInSupprimer.DistinguishedName -TargetPath $UtilisateurOU

        # Reinitialisation et reactivation de l'utilisateur
        Set-ADUser -Identity $UtilisateurMatricule `
            -Title "$UtilisateurFonction" `
            -Department "$UtilisateurBureau" `
            -Office "$UtilisateurBureau" `
            -AccountExpirationDate "$UtilisateurDate"

        Set-ADAccountPassword $UtilisateurMatricule -Reset -NewPassword (ConvertTo-SecureString -AsPlainText
"Progr00++" -Force)

        Set-ADUser $UtilisateurMatricule -ChangePasswordAtLogon $true
        Set-ADUser -Identity $UtilisateurMatricule -Enabled $true
        Set-ADUser -Identity $UtilisateurMatricule -SmartcardLogonRequired $false
    }
}

```

```

    Info "L'utilisateur $UtilisateurMatricule a ete reactive, mis a jour et deplace vers son OU normal : $UtilisateurOU."
    $MatriculesTraites += $UtilisateurMatricule
} catch {
    Error " Impossible de reactiver ou deplacer l'utilisateur $UtilisateurMatricule : $_ "
    $erreursTrouvees = $true
    continue
}
}

# Si l'utilisateur n'existe pas, creation de l'utilisateur
else {
    # Verification de l'existence de l'OU
    if (-not (Get-ADOrganizationalUnit -LDAPFilter "(distinguishedName=$UtilisateurOU)" -ErrorAction SilentlyContinue)) {
        Error " Ligne $index : L'OU '$UtilisateurOU' n'existe pas. L'execution du script est arrêtee."
        Exit 1
    }

    # Creation de l'utilisateur dans l'OU specifiee ou dans 'OU=A_attribuer'
    Info "Creation de l'utilisateur dans l'OU : $UtilisateurOU"

    New-ADUser -GivenName "$UtilisateurPrenom" -Surname "$UtilisateurNom" -Name "$UtilisateurNom
$UtilisateurPrenom" `
        -DisplayName "$UtilisateurNom $UtilisateurPrenom" -SamAccountName "$UtilisateurMatricule" `
        -Path $UtilisateurOU -UserPrincipalName "$UtilisateurPN" -Title "$UtilisateurFonction" `
        -Department "$UtilisateurBureau" -Office "$UtilisateurBureau" -EmployeeID "$UtilisateurAGRIH"
    Info "Utilisateur $UtilisateurMatricule cree dans $UtilisateurOU"

    # Reinitialisation du mot de passe de l'utilisateur
    Set-ADAccountPassword $UtilisateurMatricule -Reset -NewPassword (ConvertTo-SecureString -AsPlainText "Progr00++"
-Force)
    Info "Modification de mot de passe pour $UtilisateurNom $UtilisateurPrenom effectuee"

    # Demande de changement de mot de passe a la premiere connexion
    Set-ADUser $UtilisateurMatricule -ChangePasswordAtLogon $True
    Info "Changement de mot de passe a la premiere connexion Obligatoire"

```

#Si le champ dossier n'est pas vide, ecrire dans un fichier les commandes de creations de dossier, partage et droits dans un fichier

```
if ([string]::IsNullOrEmpty($UtilisateurDossier)){  
    Warning "le champ dossier est vide"  
    else{  
        fichier "New-Item -Path 'E:\Users\$UtilisateurDossier\$UtilisateurMatricule' -ItemType Directory"  
        fichier "New-SmbShare -Name '$UtilisateurMatricule' -Path 'E:\Users\$UtilisateurDossier\$UtilisateurMatricule'  
-ChangeAccess 'Tout le monde'"  
        fichier "Add-NTFSAccess -Path 'E:\Users\$UtilisateurDossier\$UtilisateurMatricule' -Account '$UtilisateurMatricule'  
-AccessRights Modify"  
        Set-ADUser -Identity "$UtilisateurMatricule" `  
            -HomeDirectory "\\chausers\$UtilisateurMatricule$" `  
            -HomeDrive "G:" `  
            -ScriptPath "charras.vbs"  
        Info "Ajout effectuer avec succes au fichier Commandes-CHAUSERS-$formattedDate.txt"
```

#Si le champ BDD n'est pas vide, ecrire dans un fichier les commandes de creations de boite mail et du quotas

```
if ([string]::IsNullOrEmpty($UtilisateurBDD)){  
    Warning "le champ BDD est vide"  
    else{  
        mail "Enable-Mailbox -Identity '$UtilisateurMatricule' -Alias '$UtilisateurPrenom.$UtilisateurNom' -Database  
'$UtilisateurBDD'"  
        mail "Set-Mailbox -Identity '$UtilisateurMatricule' -IssueWarningQuota 1536mb -ProhibitSendQuota 1945mb  
-ProhibitSendReceiveQuota 2gb -UseDatabaseQuotaDefaults $false"  
        Info "Ajout effectuer avec succes au fichier Commandes-CHAEXCH-$formattedDate.txt" }
```

Ajout de l'utilisateur a des groupes specifiques

```
Add-ADGroupMember -Identity $Groupemdp -Members "$UtilisateurMatricule"
```

```
Info "Ajout effectue avec succes a la strategie de mot de passe"
```

Ajout au groupe "Sillage" si applicable

```
if ($Sillage -eq "oui") {  
    Add-ADGroupMember -Identity $Groupesillage -Members "$UtilisateurMatricule"  
    Info "Ajout effectue avec succes a Sillage pour $UtilisateurNom $UtilisateurPrenom."  
}
```

```

# Ajout au groupe "CHA" si applicable
if (-not [string]::IsNullOrEmpty($UtilisateurPN)) {
    Add-ADGroupMember -Identity $Groupecha -Members "$UtilisateurMatricule"
    Info "Ajout effectue avec succes a Tout le CHA pour $UtilisateurNom $UtilisateurPrenom."
}

# Gestion des groupes pour les medecins et internes
if ($UtilisateurGrade -eq "(Dr)") {
    Add-ADGroupMember -Identity $Groupemedecin -Members "$UtilisateurMatricule"
    Info "Ajout effectue avec succes au groupe Medecin CHA"
}

if ($UtilisateurGrade -eq "(Int)") {
    Add-ADGroupMember -Identity $GroupeInternesCHA -Members "$UtilisateurMatricule"
    Info "Ajout effectue avec succes au groupe Internes CHA"
}

# Verification de la date d'expiration
if (-not [string]::IsNullOrEmpty($UtilisateurDate)) {
    try {
        $ParsedDate = [datetime]::ParseExact($UtilisateurDate, "dd/MM/yyyy", $null)
        Set-ADAccountExpiration -Identity $UtilisateurMatricule -DateTime $ParsedDate
        Info "Date d'expiration ajoutee : $ParsedDate"
    } catch {
        Error "Format de date invalide pour $UtilisateurNom $UtilisateurPrenom : $UtilisateurDate"
    }
} else {
    Warning "Aucune date d'expiration n'a ete donnee pour $UtilisateurNom $UtilisateurPrenom."
}

# Verification des informations utilisateur
$MatriculesTraites += $UtilisateurMatricule

```

```

    # Activation de l'utilisateur
    Enable-ADAccount "$UtilisateurMatricule"
}

# Reinitialisation des variables apres traitement de l'utilisateur
$UtilisateurNom = $null
$UtilisateurPrenom = $null
$UtilisateurMatricule = $null
$UtilisateurAGRIH = $null
$UtilisateurBureau = $null
$UtilisateurFonction = $null
$UtilisateurOU = $null
$UtilisateurPath = $null
$UtilisateurDossier = $null
$UtilisateurBDD = $null
$UtilisateurDate = $null
$UtilisateurGrade = $null
$Groupe = $null
$Sillage = $null
$UtilisateurPN = $null
}

if ($MatriculesTraites.Count -gt 0) {
    Write-Host "`nResume des utilisateurs traites :`n" -ForegroundColor Cyan

    $Resultats = @()

    foreach ($matricule in $MatriculesTraites) {
        try {
            $user = Get-ADUser -Identity $matricule -Properties * |
                Select-Object SamAccountName, CN, EmployeeID, EmailAddress,
                    AccountExpirationDate, Department, Title, Enabled,
                    Created, ScriptPath, HomeDrive, HomeDirectory, CanonicalName

```

```

        $Resultats += $user
    } catch {
        Write-Warning "Impossible de recuperer les infos pour l'utilisateur $matricule : $_"
    }
}

if ($Resultats.Count -gt 0) {
    $Resultats | Format-Table -Wrap -AutoSize | Out-String -Width 500
    $Resultats | Export-Csv -Path "C:\Scripts\Controle-$formattedDate.txt" -NoTypeInfo -Encoding UTF8
}

}

} else {
    Write-Host "❌ Creation des utilisateurs annulee."
}

```

Script de mappage automatique de lecteurs :

```

$CSVFile = "\\TEST\Script\compte.csv"
$Date = Get-Date -Format "yyyy-MM-dd HH:mm:ss"
$CurrentUserMatricule = $env:USERNAME

try {
    $CSVData = Import-CSV -Path $CSVFile -Delimiter ";" -Encoding UTF8
}
catch {
    Add-Content -Path $LogFile -Value "$Date [ERROR] [$CurrentUserMatricule] Échec lors de l'import du fichier CSV : $_"
    exit
}

$UserFound = $false

foreach ($Utilisateur in $CSVData) {
    $UtilisateurMatricule = $Utilisateur.Matricule1

    if ($UtilisateurMatricule -eq $CurrentUserMatricule) {
        $UserFound = $true
        $UtilisateurExchange = $Utilisateur.Dossier

        try {
            New-PSDrive -Name G -Root "\\TEST\partages\$UtilisateurExchange\$UtilisateurMatricule" -PSProvider FileSystem
        } catch {
            Write-Warning "Erreur lors de la création du lecteur G pour l'utilisateur $UtilisateurMatricule"
        }
    }
}

if ($UserFound) {
    Write-Host "Lecteur G créé avec succès"
} else {
    Write-Host "Lecteur G n'a pas été créé"
}

```

```

        Add-Content -Path $LogFile -Value "$Date [INFO] [$CurrentUserMatricule] Lecteur G: monté depuis
\\TEST\partages\${UtilisateurExchange}\${UtilisateurMatricule}"
    }
    catch {
        Add-Content -Path $LogFile -Value "$Date [ERROR] [$CurrentUserMatricule] Échec du montage du lecteur G: : $_"
    }

    break
}
}

if (-not $UserFound) {
    Add-Content -Path $LogFile -Value "$Date [WARN] [$CurrentUserMatricule] Utilisateur non trouvé dans le fichier CSV"
}

```

Graylog :

Succès de l'audit	22/04/2025 15:15:47	Microsoft Windows security auditing.	4720	User Account Management
Succès de l'audit	22/04/2025 15:15:38	Microsoft Windows security auditing.	4624	Logon
Succès de l'audit	22/04/2025 15:15:37	Microsoft Windows security auditing.	4726	User Account Management
Succès de l'audit	22/04/2025 15:15:37	Microsoft Windows security auditing.	4726	User Account Management
Succès de l'audit	22/04/2025 15:15:37	Microsoft Windows security auditing.	4726	User Account Management
Succès de l'audit	22/04/2025 15:15:37	Microsoft Windows security auditing.	4726	User Account Management
Succès de l'audit	22/04/2025 15:15:37	Microsoft Windows security auditing.	4726	User Account Management

Événement 4720, Microsoft Windows security auditing.

Général Détails

Un compte d'utilisateur a été créé.

Visualisation ID AD

Serveur CDR :

Fonction fetch_calls_by_datetime_range :

```
def fetch_calls_by_datetime_range(start_datetime, end_datetime, calling_party_numbers=None,
called_party_numbers=None):
```

```
    """Cherche les appels entre deux dates et heures."""
```

```
    conn = mysql.connector.connect(**DB_CONFIG)
```

```
    cursor = conn.cursor()
```

```
    # Base de la requête
```

```
    query = """
```

```
        SELECT
```

```
            d.value AS date,
```

```
            cp.value AS appelant,
```

```
            op.value AS appele,
```

```
            du.value AS duree
```

```
        FROM
```

```
            dateTimeOrigination d
```

```
        JOIN
```

```
            callingPartyNumber cp ON d.id = cp.id
```

```
        JOIN
```

```
            originalCalledPartyNumber op ON d.id = op.id
```

```
        JOIN
```

```
            duration du ON d.id = du.id
```

```
        WHERE
```

```
            d.value BETWEEN %s AND %s
```

```

"""

# Ajouter des conditions pour les numéros d'appelant et d'appelé si fournis
params = [start_datetime, end_datetime]

if calling_party_numbers:
    query += " AND cp.value IN (%s)" % ', '.join(['%s' * len(calling_party_numbers)])
    params.extend(calling_party_numbers)

if called_party_numbers:
    query += " AND op.value IN (%s)" % ', '.join(['%s' * len(called_party_numbers)])
    params.extend(called_party_numbers)

cursor.execute(query, params)

result = cursor.fetchall()

conn.close()

return result

```

Fonction fetch_id :

```

def fetch_id_by_calling_party_number(calling_party_number):
    """Cherche l'ID dans la table callingPartyNumber pour le numéro donné."""
    conn = mysql.connector.connect(**DB_CONFIG)
    cursor = conn.cursor()

    # Recherche de l'ID dans la table `callingPartyNumber`
    query = "SELECT id FROM callingPartyNumber WHERE value = %s"
    cursor.execute(query, (calling_party_number,))

    result = cursor.fetchone() # Récupère une seule ligne

    conn.close()

    if result:
        return result[0] # Retourne l'ID si trouvé
    else:
        return None # Aucun résultat trouvé

```

fetch_all_data_id :

```

def fetch_all_data_for_id(id_value):
    """Cherche les données correspondant à l'ID dans toutes les tables."""
    conn = mysql.connector.connect(**DB_CONFIG)
    cursor = conn.cursor()

```

```

# Récupère la liste de toutes les tables dans la base de données
cursor.execute("SHOW TABLES")
tables = cursor.fetchall()

data = {}

# Rechercher dans chaque table pour l'ID
for (table,) in tables:
    try:
        # On fait une recherche dans chaque table pour cet ID, en excluant la colonne `id`
        query = f"SELECT * FROM {table} WHERE id = %s"
        cursor.execute(query, (id_value,))
        result = cursor.fetchall() # Récupère toutes les lignes correspondant à cet ID
        if result:
            # Exclure la colonne `id` des résultats
            data[table] = [row[1:] for row in result] # Supprime la première colonne (id)
    except mysql.connector.Error as err:
        print(f"Erreur lors de l'accès à la table {table}: {err}")

conn.close()

return data

```